After verifying that everything on the test machine is working properly, use the same procedure to install the new software on each of the other machines in the build set.

The same methodology can be used for the ports tree. The first step is to share /usr/ports via NFS to all the machines in the build set. To configure /etc/make.conf to share distfiles, set `DISTDIR` to a common shared directory that is writable by whichever user `root` is mapped to by the NFS mount. Each machine should set `WRKDIRPREFIX` to a local build directory, if ports are to be built locally. Alternately, if the build system is to build and distribute packages to the machines in the build set, set `PACKAGES` on the build system to a directory similar to `DISTDIR`.

26.9. Building on non-FreeBSD Hosts

Historically, building required a FreeBSD host. Nowadays, FreeBSD can be built on Linux and macOS.

To build on a non-FreeBSD host, the `tools/build/make.py` script is recommended. This script acts as a wrapper around `bmake`, which is the make implementation used by FreeBSD. It ensures that the necessary tooling, including the actual FreeBSD's make(1), is bootstrapped and that the build environment is properly configured. In particular, it sets the external toolchain variables, such as `XCC`, `XLD`, and others. Additionally, the script can pass any additional command arguments, such as `-j 4` for parallel builds or specific make targets, to `bmake`.



A recent version of `bmake` can be used instead of the `tools/build/make.py` script as well. In that case, however, required environment variables need to be set manually (the easiest way to obtain a list of them is by running `tools/build/make.py --debug`).

Otherwise, the list of prerequisites for building FreeBSD is rather short. In fact, it boils down to installing a couple of dependencies.

On macOS, the only dependency is LLVM. The necessary dependencies can be installed with package manager (e.g., Homebrew):

```
brew install llvm
```

On a Linux distributions, install Clang version 10.0 or newer and the headers for libarchive and libbz2 (often packaged as libarchive-dev and libbz2-dev).

Once the dependencies are installed, the host should be able to build FreeBSD.

For example, the following `tools/build/make.py` invocation builds the world:

```
MAKEOBJDIRPREFIX=/tmp/obj tools/build/make.py -j 8 TARGET=arm64 TARGET_ARCH=aarch64
buildworld
```

It builds the world for target `aarch64:arm64` on 8 CPUs and uses /tmp/obj for object files. Note that the variables `MAKEOBJDIRPREFIX`, `TARGET`, and `TARGET_ARCH` are mandatory when building on non-

FreeBSD hosts. Also, make sure to create the object directory pointed to by the `MAKEOBJDIRPREFIX` environment variable.

Refer to arch(7) and build(7) for more details.

Chapter 27. DTrace

27.1. Synopsis

DTrace, also known as Dynamic Tracing, was developed by Sun™ as a tool for locating performance bottlenecks in production and pre-production systems. In addition to diagnosing performance problems, DTrace can be used to help investigate and debug unexpected behavior in both the FreeBSD kernel and in userland programs.

DTrace is a remarkable profiling tool, with an impressive array of features for diagnosing system issues. It may also be used to run pre-written scripts to take advantage of its capabilities. Users can author their own utilities using the DTrace D Language, allowing them to customize their profiling based on specific needs.

The FreeBSD implementation provides full support for kernel DTrace and experimental support for userland DTrace. Userland DTrace allows users to perform function boundary tracing for userland programs using the `pid` provider, and to insert static probes into userland programs for later tracing. Some ports, such as databases/postgresql12-server and lang/php74 have a DTrace option to enable static probes.

The official guide to DTrace is maintained by the illumos project at illumos Dynamic Tracing Guide.

Read this chapter to learn:

- What DTrace is and what features it provides.
- Differences between the Solaris™ DTrace implementation and the one provided by FreeBSD.
- How to enable and use DTrace on FreeBSD.

Before reading this chapter:

- Understand UNIX® and FreeBSD basics (FreeBSD Basics).
- Have some familiarity with security and how it pertains to FreeBSD (Security).

27.2. Implementation Differences

While the DTrace in FreeBSD is similar to that found in Solaris™, differences do exist. The primary difference is that in FreeBSD, DTrace is implemented as a set of kernel modules and DTrace can not be used until the modules are loaded. To load all of the necessary modules:

```
# kldload dtraceall
```

Beginning with FreeBSD 10.0-RELEASE, the modules are automatically loaded when dtrace(1) is run.

FreeBSD uses the `DDB_CTF` kernel option to enable support for loading ctf(5) data from kernel modules and the kernel itself. `CTF` is the Solaris™ Compact C Type Format which encapsulates a

reduced form of debugging information similar to DWARF and the venerable stabs. CTF data is added to binaries by the ctfconvert(1) and ctfmerge(1) build tools. The ctfconvert utility parses DWARF ELF debug sections created by the compiler and ctfmerge merges CTF ELF sections from objects into either executables or shared libraries.

Some different providers exist for FreeBSD than for Solaris™. Most notable is the dtmalloc provider, which allows tracing malloc(9) by type in the FreeBSD kernel. Some of the providers found in Solaris™, such as cpc, are not present in FreeBSD. These may appear in future versions of FreeBSD. Moreover, some of the providers available in both operating systems are not compatible, in the sense that their probes have different argument types. Thus, D scripts written on Solaris™ may or may not work unmodified on FreeBSD, and vice versa.

Due to security differences, only root may use DTrace on FreeBSD. Solaris™ has a few low level security checks which do not yet exist in FreeBSD. As such, the /dev/dtrace/dtrace is strictly limited to root.

DTrace falls under the Common Development and Distribution License (CDDL) license. To view this license on FreeBSD, see /usr/src/cddl/contrib/opensolaris/OPENSOLARIS.LICENSE or view it online at http://opensource.org/licenses/CDDL-1.0. While a FreeBSD kernel with DTrace support is BSD licensed, the CDDL is used when the modules are distributed in binary form or the binaries are loaded.

27.3. Enabling DTrace Support

In FreeBSD 9.2 and 10.0, DTrace support is built into the GENERIC kernel. Users of earlier versions of FreeBSD or who prefer to statically compile in DTrace support should add the following lines to a custom kernel configuration file and recompile the kernel using the instructions in Configuring the FreeBSD Kernel:

```
options         KDTRACE_HOOKS
options         DDB_CTF
makeoptions DEBUG=-g
makeoptions WITH_CTF=1
```

Users of the AMD64 architecture should also add this line:

```
options         KDTRACE_FRAME
```

This option provides support for dtrace_fbt(4). While DTrace will work without this option, there will be limited support for function boundary tracing.

Once the FreeBSD system has rebooted into the new kernel, or the DTrace kernel modules have been loaded using kldload dtraceall, install the current DTrace Toolkit (sysutils/dtrace-toolkit), a collection of ready-made scripts for collecting system information. There are scripts to check open files, memory, CPU usage, and a lot more. FreeBSD includes some scripts in the base system as well; see /usr/share/dtrace.



The scripts found in /usr/share/dtrace have been specifically ported to FreeBSD. Not all of the scripts found in the DTrace Toolkit will work as-is on FreeBSD and some scripts may require some effort in order for them to work on FreeBSD.

The DTrace Toolkit includes many scripts in the special language of DTrace. This language is called the D language and it is very similar to C++. An in depth discussion of the language is beyond the scope of this document. Refer to the FreeBSD d(7) manual page for an overview of the D language on FreeBSD. The D language is also covered extensively in the illumos Dynamic Tracing Guide.

27.4. Enabling DTrace in Out-of-Kernel Modules

To add DTrace support to an out-of-kernel module, which is useful for development and debugging, include the following line in the module's Makefile:

```
CFLAGS+= -DKDTRACE_HOOKS
```

This flag enables DTrace hooks during compilation, allowing for advanced debugging and monitoring of the module. Ensure to recompile the module after this modification to activate DTrace functionality.

27.5. Using DTrace

DTrace scripts consist of a list of one or more *probes*, or instrumentation points, where each probe is associated with an action. Whenever the condition for a probe is met, the associated action is executed. For example, an action may occur when a file is opened, a process is started, or a line of code is executed. The action might be to log some information or to modify context variables. The reading and writing of context variables allows probes to share information and to cooperatively analyze the correlation of different events.

To view all probes, the administrator can execute the following command:

```
# dtrace -l | more
```

Each probe has an `ID`, a `PROVIDER` (e.g., `dtrace` or `fbt`), a `MODULE`, and a `FUNCTION NAME`. Refer to dtrace(1) for more information about this command.

The examples in this section provide an overview of how to use two of the fully supported scripts from the DTrace Toolkit: the hotkernel and procsystime scripts.

The hotkernel script is designed to identify which function is using the most kernel time. It will produce output similar to the following:

```
# cd /usr/local/share/dtrace-toolkit
# ./hotkernel
Sampling... Hit Ctrl-C to end.
```

As instructed, use the `Ctrl` + `C` key combination to stop the process. Upon termination, the script will display a list of kernel functions and timing information, sorting the output in increasing order of time:

```
kernel`_thread_lock_flags                                2   0.0%
0xc1097063                                               2   0.0%
kernel`sched_userret                                     2   0.0%
kernel`kern_select                                       2   0.0%
kernel`generic_copyin                                    3   0.0%
kernel`_mtx_assert                                       3   0.0%
kernel`vm_fault                                          3   0.0%
kernel`sopoll_generic                                    3   0.0%
kernel`fixup_filename                                    4   0.0%
kernel`_isitmyx                                          4   0.0%
kernel`find_instance                                     4   0.0%
kernel`_mtx_unlock_flags                                 5   0.0%
kernel`syscall                                           5   0.0%
kernel`DELAY                                             5   0.0%
0xc108a253                                               6   0.0%
kernel`witness_lock                                      7   0.0%
kernel`read_aux_data_no_wait                             7   0.0%
kernel`Xint0x80_syscall                                  7   0.0%
kernel`witness_checkorder                                7   0.0%
kernel`sse2_pagezero                                     8   0.0%
kernel`strncmp                                           9   0.0%
kernel`spinlock_exit                                    10   0.0%
kernel`_mtx_lock_flags                                  11   0.0%
kernel`witness_unlock                                   15   0.0%
kernel`sched_idletd                                    137   0.3%
0xc10981a5                                           42139  99.3%
```

This script will also work with kernel modules. To use this feature, run the script with `-m`:

```
# ./hotkernel -m
Sampling... Hit Ctrl-C to end.
^C
MODULE                                                COUNT   PCNT
0xc107882e                                                1   0.0%
0xc10e6aa4                                                1   0.0%
0xc1076983                                                1   0.0%
0xc109708a                                                1   0.0%
0xc1075a5d                                                1   0.0%
0xc1077325                                                1   0.0%
0xc108a245                                                1   0.0%
0xc107730d                                                1   0.0%
0xc1097063                                                2   0.0%
0xc108a253                                               73   0.0%
kernel                                                  874   0.4%
```

```
0xc10981a5                                          213781  99.6%
```

The procsystime script captures and prints the system call time usage for a given process ID (PID) or process name. In the following example, a new instance of /bin/csh was spawned. Then, procsystime was executed and remained waiting while a few commands were typed on the other incarnation of csh. These are the results of this test:

```
# ./procsystime -n csh
Tracing... Hit Ctrl-C to end...
^C

Elapsed Times for processes csh,

         SYSCALL          TIME (ns)
          getpid              6131
       sigreturn              8121
           close             19127
           fcntl             19959
             dup             26955
         setpgid             28070
            stat             31899
       setitimer             40938
           wait4             62717
       sigaction             67372
     sigprocmask            119091
    gettimeofday            183710
           write            263242
          execve            492547
           ioctl            770073
           vfork           3258923
      sigsuspend           6985124
            read        3988049784
```

As shown, the read(2) system call used the most time in nanoseconds while the getpid(2) system call used the least amount of time.

Chapter 28. USB Device Mode / USB OTG

28.1. Synopsis

This chapter covers the use of USB Device Mode and USB On The Go (USB OTG) in FreeBSD. This includes virtual serial consoles, virtual network interfaces, and virtual USB drives.

When running on hardware that supports USB device mode or USB OTG, like that built into many embedded boards, the FreeBSD USB stack can run in *device mode*. Device mode makes it possible for the computer to present itself as different kinds of USB device classes, including serial ports, network adapters, and mass storage, or a combination thereof. A USB host like a laptop or desktop computer is able to access them just like physical USB devices. Device mode is sometimes called the "USB gadget mode".

There are two basic ways the hardware can provide the device mode functionality: with a separate "client port", which only supports the device mode, and with a USB OTG port, which can provide both device and host mode. For USB OTG ports, the USB stack switches between host-side and device-side automatically, depending on what is connected to the port. Connecting a USB device like a memory stick to the port causes FreeBSD to switch to host mode. Connecting a USB host like a computer causes FreeBSD to switch to device mode. Single purpose "client ports" always work in device mode.

What FreeBSD presents to the USB host depends on the `hw.usb.template` sysctl. Some templates provide a single device, such as a serial terminal; others provide multiple ones, which can all be used at the same time. An example is the template 10, which provides a mass storage device, a serial console, and a network interface. See usb_template(4) for the list of available values.

Note that in some cases, depending on the hardware and the hosts operating system, for the host to notice the configuration change, it must be either physically disconnected and reconnected, or forced to rescan the USB bus in a system-specific way. When FreeBSD is running on the host, usbconfig(8) `reset` can be used. This also must be done after loading usb_template.ko if the USB host was already connected to the USBOTG socket.

Read this chapter to learn:

- How to set up USB Device Mode functionality on FreeBSD.
- How to configure the virtual serial port on FreeBSD.
- How to connect to the virtual serial port from various operating systems.
- How to configure FreeBSD to provide a virtual USB network interface.
- How to configure FreeBSD to provide a virtual USB storage device.

28.2. USB Virtual Serial Ports

28.2.1. Configuring USB Device Mode Serial Ports

Virtual serial port support is provided by templates number 3, 8, and 10. Note that template 3 works

with Microsoft Windows 10 without the need for special drivers and INF files. Other host operating systems work with all three templates. Both usb_template(4) and umodem(4) kernel modules must be loaded.

To enable USB device mode serial ports, add those lines to /etc/ttys:

```
ttyU0   "/usr/libexec/getty 3wire"  vt100   onifconsole secure
ttyU1   "/usr/libexec/getty 3wire"  vt100   onifconsole secure
```

Then add these lines to /etc/devd.conf:

```
notify 100 {
    match "system"      "DEVFS";
    match "subsystem"   "CDEV";
    match "type"        "CREATE";
    match "cdev"        "ttyU[0-9]+";
    action "/sbin/init q";
};
```

Reload the configuration if devd(8) is already running:

```
# service devd restart
```

Make sure the necessary modules are loaded and the correct template is set at boot by adding those lines to /boot/loader.conf, creating it if it does not already exist:

```
umodem_load="YES"
hw.usb.template=3
```

To load the module and set the template without rebooting use:

```
# kldload umodem
# sysctl hw.usb.template=3
```

28.2.2. Connecting to USB Device Mode Serial Ports from FreeBSD

To connect to a board configured to provide USB device mode serial ports, connect the USB host, such as a laptop, to the boards USB OTG or USB client port. Use `pstat -t` on the host to list the terminal lines. Near the end of the list a USB serial port is shown, e.g. "ttyU0". To open the connection, use:

```
# cu -l /dev/ttyU0
```

After pressing the `Enter` key a few times a login prompt appears.

28.2.3. Connecting to USB Device Mode Serial Ports from macOS

To connect to a board configured to provide USB device mode serial ports, connect the USB host, such as a laptop, to the boards USB OTG or USB client port. To open the connection, use:

```
# cu -l /dev/cu.usbmodemFreeBSD1
```

28.2.4. Connecting to USB Device Mode Serial Ports from Linux

To connect to a board configured to provide USB device mode serial ports, connect the USB host, such as a laptop, to the boards USB OTG or USB client port. To open the connection, use:

```
# minicom -D /dev/ttyACM0
```

28.2.5. Connecting to USB Device Mode Serial Ports from Microsoft Windows 10

To connect to a board configured to provide USB device mode serial ports, connect the USB host, such as a laptop, to the boards USB OTG or USB client port. To open a connection a serial terminal program is needed, such as PuTTY. To check the COM port name used by Windows, run Device Manager, expand "Ports (COM & LPT)". A name similar to "USB Serial Device (COM4)" appears. Run the serial terminal program of choice, for example PuTTY. In the PuTTY dialog set "Connection type" to "Serial", type the COMx obtained from Device Manager in the "Serial line" dialog box and click Open.

28.3. USB Device Mode Network Interfaces

Virtual network interfaces support is provided by templates number 1, 8, and 10. Note that none of them works with Microsoft Windows. Other host operating systems work with all three templates. Both usb_template(4) and if_cdce(4) kernel modules must be loaded.

Make sure the necessary modules are loaded and the correct template is set at boot by adding those lines to /boot/loader.conf, creating it if it does not already exist:

```
if_cdce_load="YES"
hw.usb.template=1
```

To load the module and set the template without rebooting use:

```
# kldload if_cdce
# sysctl hw.usb.template=1
```

28.4. USB Virtual Storage Device



The cfumass(4) driver is a USB device mode driver first available in FreeBSD 12.0.

Mass Storage target is provided by templates 0 and 10. Both usb_template(4) and cfumass(4) kernel modules must be loaded. cfumass(4) interfaces to the CTL subsystem, the same one that is used for iSCSI or Fibre Channel targets. On the host side, USB Mass Storage initiators can only access a single LUN, LUN 0.

28.4.1. Configuring USB Mass Storage Target Using the cfumass Startup Script

The simplest way to set up a read-only USB storage target is to use the cfumass rc script. To configure it this way, copy the files to be presented to the USB host machine into the `/var/cfumass` directory, and add this line to /etc/rc.conf:

```
cfumass_enable="YES"
```

To configure the target without restarting, run this command:

```
# service cfumass start
```

Differently from serial and network functionality, the template should not be set to 0 or 10 in /boot/loader.conf. This is because the LUN must be set up before setting the template. The cfumass startup script sets the correct template number automatically when started.

28.4.2. Configuring USB Mass Storage Using Other Means

The rest of this chapter provides detailed description of setting the target without using the cfumass rc file. This is necessary if e.g. one wants to provide a writeable LUN.

USB Mass Storage does not require the ctld(8) daemon to be running, although it can be used if desired. This is different from iSCSI. Thus, there are two ways to configure the target: ctladm(8), or ctld(8). Both require the cfumass.ko kernel module to be loaded. The module can be loaded manually:

```
# kldload cfumass
```

If cfumass.ko has not been built into the kernel, /boot/loader.conf can be set to load the module at boot:

```
cfumass_load="YES"
```

A LUN can be created without the ctld(8) daemon:

```
# ctladm create -b block -o file=/data/target0
```

This presents the contents of the image file /data/target0 as a LUN to the USB host. The file must exist before executing the command. To configure the LUN at system startup, add the command to /etc/rc.local.

ctld(8) can also be used to manage LUNs. Create /etc/ctl.conf, add a line to /etc/rc.conf to make sure ctld(8) is automatically started at boot, and then start the daemon.

This is an example of a simple /etc/ctl.conf configuration file. Refer to ctl.conf(5) for a complete description of the options.

```
target naa.50015178f369f092 {
    lun 0 {
        path /data/target0
        size 4G
    }
}
```

The example creates a single target with a single LUN. The `naa.50015178f369f092` is a device identifier composed of 32 random hexadecimal digits. The `path` line defines the full path to a file or zvol backing the LUN. That file must exist before starting ctld(8). The second line is optional and specifies the size of the LUN.

To make sure the ctld(8) daemon is started at boot, add this line to /etc/rc.conf:

```
ctld_enable="YES"
```

To start ctld(8) now, run this command:

```
# service ctld start
```

As the ctld(8) daemon is started, it reads /etc/ctl.conf. If this file is edited after the daemon starts, reload the changes so they take effect immediately:

```
# service ctld reload
```

Part IV: Network Communication

FreeBSD is one of the most widely deployed operating systems for high performance network servers. The chapters in this part cover:

- Serial communication
- PPP and PPP over Ethernet
- Electronic Mail
- Running Network Servers
- Firewalls
- Other Advanced Networking Topics

These chapters are designed to be read when the information is needed. They do not need to be read in any particular order, nor is it necessary to read all of them before using FreeBSD in a network environment.

Chapter 29. Serial Communications

29.1. Synopsis

UNIX® has always had support for serial communications as the very first UNIX® machines relied on serial lines for user input and output. Things have changed a lot from the days when the average terminal consisted of a 10-character-per-second serial printer and a keyboard. This chapter covers some of the ways serial communications can be used on FreeBSD.

Read this chapter to learn:

- How to connect terminals to a FreeBSD system.
- How to use a modem to dial out to remote hosts.
- How to allow remote users to login to a FreeBSD system with a modem.
- How to boot a FreeBSD system from a serial console.

Before reading this chapter:

- Know how to configure and install a custom kernel.
- Understand FreeBSD permissions and processes.
- Have access to the technical manual for the serial hardware to be used with FreeBSD.

29.2. Serial Terminology and Hardware

The following terms are often used in serial communications:

bps
: Bits per Second (bps) is the rate at which data is transmitted.

DTE
: Data Terminal Equipment (DTE) is one of two endpoints in a serial communication. An example would be a computer.

DCE
: Data Communications Equipment (DCE) is the other endpoint in a serial communication. Typically, it is a modem or serial terminal.

RS-232
: The original standard which defined hardware serial communications. It has since been renamed to TIA-232.

When referring to communication data rates, this section does not use the term *baud*. Baud refers to the number of electrical state transitions made in a period of time, while bps is the correct term to use.

To connect a serial terminal to a FreeBSD system, a serial port on the computer and the proper

cable to connect to the serial device are needed. Users who are already familiar with serial hardware and cabling can safely skip this section.

29.2.1. Serial Cables and Ports

There are several different kinds of serial cables. The two most common types are null-modem cables and standard RS-232 cables. The documentation for the hardware should describe the type of cable required.

These two types of cables differ in how the wires are connected to the connector. Each wire represents a signal, with the defined signals summarized in RS-232C Signal Names. A standard serial cable passes all of the RS-232C signals straight through. For example, the "Transmitted Data" pin on one end of the cable goes to the "Transmitted Data" pin on the other end. This is the type of cable used to connect a modem to the FreeBSD system, and is also appropriate for some terminals.

A null-modem cable switches the "Transmitted Data" pin of the connector on one end with the "Received Data" pin on the other end. The connector can be either a DB-25 or a DB-9.

A null-modem cable can be constructed using the pin connections summarized in DB-25 to DB-25 Null-Modem Cable, DB-9 to DB-9 Null-Modem Cable, and DB-9 to DB-25 Null-Modem Cable. While the standard calls for a straight-through pin 1 to pin 1 "Protective Ground" line, it is often omitted. Some terminals work using only pins 2, 3, and 7, while others require different configurations. When in doubt, refer to the documentation for the hardware.

Table 41. RS-232C Signal Names

Acronyms	Names
RD	Received Data
TD	Transmitted Data
DTR	Data Terminal Ready
DSR	Data Set Ready
DCD	Data Carrier Detect
SG	Signal Ground
RTS	Request to Send
CTS	Clear to Send

Table 42. DB-25 to DB-25 Null-Modem Cable

Signal	Pin #		Pin #	Signal
SG	7	connects to	7	SG
TD	2	connects to	3	RD
RD	3	connects to	2	TD
RTS	4	connects to	5	CTS
CTS	5	connects to	4	RTS

Signal	Pin #		Pin #	Signal
DTR	20	connects to	6	DSR
DTR	20	connects to	8	DCD
DSR	6	connects to	20	DTR
DCD	8	connects to	20	DTR

Table 43. DB-9 to DB-9 Null-Modem Cable

Signal	Pin #		Pin #	Signal
RD	2	connects to	3	TD
TD	3	connects to	2	RD
DTR	4	connects to	6	DSR
DTR	4	connects to	1	DCD
SG	5	connects to	5	SG
DSR	6	connects to	4	DTR
DCD	1	connects to	4	DTR
RTS	7	connects to	8	CTS
CTS	8	connects to	7	RTS

Table 44. DB-9 to DB-25 Null-Modem Cable

Signal	Pin #		Pin #	Signal
RD	2	connects to	2	TD
TD	3	connects to	3	RD
DTR	4	connects to	6	DSR
DTR	4	connects to	8	DCD
SG	5	connects to	7	SG
DSR	6	connects to	20	DTR
DCD	1	connects to	20	DTR
RTS	7	connects to	5	CTS
CTS	8	connects to	4	RTS



When one pin at one end connects to a pair of pins at the other end, it is usually implemented with one short wire between the pair of pins in their connector and a long wire to the other single pin.

Serial ports are the devices through which data is transferred between the FreeBSD host computer and the terminal. Several kinds of serial ports exist. Before purchasing or constructing a cable, make sure it will fit the ports on the terminal and on the FreeBSD system.

Most terminals have DB-25 ports. Personal computers may have DB-25 or DB-9 ports. A multiport serial card may have RJ-12 or RJ-45/ ports. See the documentation that accompanied the hardware for specifications on the kind of port or visually verify the type of port.

In FreeBSD, each serial port is accessed through an entry in /dev. There are two different kinds of entries:

- Call-in ports are named /dev/ttyuN where *N* is the port number, starting from zero. If a terminal is connected to the first serial port (COM1), use /dev/ttyu0 to refer to the terminal. If the terminal is on the second serial port (COM2), use /dev/ttyu1, and so forth. Generally, the call-in port is used for terminals. Call-in ports require that the serial line assert the "Data Carrier Detect" signal to work correctly.
- Call-out ports are named /dev/cuauN on FreeBSD versions 8.X and higher and /dev/cuadN on FreeBSD versions 7.X and lower. Call-out ports are usually not used for terminals, but are used for modems. The call-out port can be used if the serial cable or the terminal does not support the "Data Carrier Detect" signal.

FreeBSD also provides initialization devices (/dev/ttyuN.init and /dev/cuauN.init or /dev/cuadN.init) and locking devices (/dev/ttyuN.lock and /dev/cuauN.lock or /dev/cuadN.lock). The initialization devices are used to initialize communications port parameters each time a port is opened, such as `crtscts` for modems which use `RTS/CTS` signaling for flow control. The locking devices are used to lock flags on ports to prevent users or programs changing certain parameters. Refer to termios(4), uart(4), and stty(1) for information on terminal settings, locking and initializing devices, and setting terminal options, respectively.

29.2.2. Serial Port Configuration

By default, FreeBSD supports four serial ports which are commonly known as COM1, COM2, COM3, and COM4. FreeBSD also supports dumb multi-port serial interface cards, such as the BocaBoard 1008 and 2016, as well as more intelligent multi-port cards such as those made by Digiboard. However, the default kernel only looks for the standard COM ports.

To see if the system recognizes the serial ports, look for system boot messages that start with `uart`:

```
# grep uart /var/run/dmesg.boot
```

If the system does not recognize all of the needed serial ports, additional entries can be added to /boot/device.hints. This file already contains `hint.uart.0.*` entries for COM1 and `hint.uart.1.*` entries for COM2. When adding a port entry for COM3 use `0x3E8`, and for COM4 use `0x2E8`. Common IRQ addresses are `5` for COM3 and `9` for COM4.

To determine the default set of terminal I/O settings used by the port, specify its device name. This example determines the settings for the call-in port on COM2:

```
# stty -a -f /dev/ttyu1
```

System-wide initialization of serial devices is controlled by /etc/rc.d/serial. This file affects the

default settings of serial devices. To change the settings for a device, use `stty`. By default, the changed settings are in effect until the device is closed and when the device is reopened, it goes back to the default set. To permanently change the default set, open and adjust the settings of the initialization device. For example, to turn on `CLOCAL` mode, 8 bit communication, and `XON/XOFF` flow control for ttyu5, type:

```
# stty -f /dev/ttyu5.init clocal cs8 ixon ixoff
```

To prevent certain settings from being changed by an application, make adjustments to the locking device. For example, to lock the speed of ttyu5 to 57600 bps, type:

```
# stty -f /dev/ttyu5.lock 57600
```

Now, any application that opens ttyu5 and tries to change the speed of the port will be stuck with 57600 bps.

29.3. Terminals

Terminals provide a convenient and low-cost way to access a FreeBSD system when not at the computer's console or on a connected network. This section describes how to use terminals with FreeBSD.

The original UNIX® systems did not have consoles. Instead, users logged in and ran programs through terminals that were connected to the computer's serial ports.

The ability to establish a login session on a serial port still exists in nearly every UNIX®-like operating system today, including FreeBSD. By using a terminal attached to an unused serial port, a user can log in and run any text program that can normally be run on the console or in an `xterm` window.

Many terminals can be attached to a FreeBSD system. An older spare computer can be used as a terminal wired into a more powerful computer running FreeBSD. This can turn what might otherwise be a single-user computer into a powerful multiple-user system.

FreeBSD supports three types of terminals:

Dumb terminals

Dumb terminals are specialized hardware that connect to computers over serial lines. They are called "dumb" because they have only enough computational power to display, send, and receive text. No programs can be run on these devices. Instead, dumb terminals connect to a computer that runs the needed programs.

There are hundreds of kinds of dumb terminals made by many manufacturers, and just about any kind will work with FreeBSD. Some high-end terminals can even display graphics, but only certain software packages can take advantage of these advanced features.

Dumb terminals are popular in work environments where workers do not need access to

graphical applications.

Computers Acting as Terminals

Since a dumb terminal has just enough ability to display, send, and receive text, any spare computer can be a dumb terminal. All that is needed is the proper cable and some *terminal emulation* software to run on the computer.

This configuration can be useful. For example, if one user is busy working at the FreeBSD system's console, another user can do some text-only work at the same time from a less powerful personal computer hooked up as a terminal to the FreeBSD system.

There are at least two utilities in the base-system of FreeBSD that can be used to work through a serial connection: cu(1) and tip(1).

For example, to connect from a client system that runs FreeBSD to the serial connection of another system:

```
# cu -l /dev/cuauN
```

Ports are numbered starting from zero. This means that COM1 is /dev/cuau0.

Additional programs are available through the Ports Collection, such as comms/minicom.

X Terminals

X terminals are the most sophisticated kind of terminal available. Instead of connecting to a serial port, they usually connect to a network like Ethernet. Instead of being relegated to text-only applications, they can display any Xorg application.

This chapter does not cover the setup, configuration, or use of X terminals.

29.3.1. Terminal Configuration

This section describes how to configure a FreeBSD system to enable a login session on a serial terminal. It assumes that the system recognizes the serial port to which the terminal is connected and that the terminal is connected with the correct cable.

In FreeBSD, `init` reads /etc/ttys and starts a `getty` process on the available terminals. The `getty` process is responsible for reading a login name and starting the `login` program. The ports on the FreeBSD system which allow logins are listed in /etc/ttys. For example, the first virtual console, ttyv0, has an entry in this file, allowing logins on the console. This file also contains entries for the other virtual consoles, serial ports, and pseudo-ttys. For a hardwired terminal, the serial port's /dev entry is listed without the `/dev` part. For example, /dev/ttyv0 is listed as `ttyv0`.

The default /etc/ttys configures support for the first four serial ports, ttyu0 through ttyu3:

```
ttyu0   "/usr/libexec/getty std.115200"   dialup  off secure
ttyu1   "/usr/libexec/getty std.115200"   dialup  off secure
ttyu2   "/usr/libexec/getty std.115200"   dialup  off secure
```

```
ttyu3   "/usr/libexec/getty std.115200"   dialup  off secure
```

When attaching a terminal to one of those ports, modify the default entry to set the required speed and terminal type, to turn the device `on` and, if needed, to change the port's `secure` setting. If the terminal is connected to another port, add an entry for the port.

Configuring Terminal Entries configures two terminals in /etc/ttys. The first entry configures a Wyse-50 connected to COM2. The second entry configures an old computer running Procomm terminal software emulating a VT-100 terminal. The computer is connected to the sixth serial port on a multi-port serial card.

Example 40. Configuring Terminal Entries

```
ttyu1  "/usr/libexec/getty std.38400"  wy50   on insecure
ttyu5   "/usr/libexec/getty std.19200"  vt100  on insecure
```

The first field specifies the device name of the serial terminal.

The second field tells `getty` to initialize and open the line, set the line speed, prompt for a user name, and then execute the `login` program. The optional *getty type* configures characteristics on the terminal line, like bps rate and parity. The available getty types are listed in /etc/gettytab. In almost all cases, the getty types that start with `std` will work for hardwired terminals as these entries ignore parity. There is a `std` entry for each bps rate from 110 to 115200. Refer to gettytab(5) for more information. When setting the getty type, make sure to match the communications settings used by the terminal. For this example, the Wyse-50 uses no parity and connects at 38400 bps. The computer uses no parity and connects at 19200 bps.

The third field is the type of terminal. For dial-up ports, `unknown` or `dialup` is typically used since users may dial up with practically any type of terminal or software. Since the terminal type does not change for hardwired terminals, a real terminal type from /etc/termcap can be specified. For this example, the Wyse-50 uses the real terminal type while the computer running Procomm is set to emulate a VT-100.

The fourth field specifies if the port should be enabled. To enable logins on this port, this field must be set to `on`.

The final field is used to specify whether the port is secure. Marking a port as `secure` means that it is trusted enough to allow `root` to login from that port. Insecure ports do not allow `root` logins. On an insecure port, users must login from unprivileged accounts and then use `su` or a similar mechanism to gain superuser privileges, as described in "The Superuser Account". For security reasons, it is recommended to change this setting to `insecure`.

After making any changes to /etc/ttys, send a SIGHUP (hangup) signal to the `init` process to force it to re-read its configuration file:

```
# kill -HUP 1
```

Since `init` is always the first process run on a system, it always has a process ID of `1`.

If everything is set up correctly, all cables are in place, and the terminals are powered up, a `getty` process should now be running on each terminal and login prompts should be available on each terminal.

29.3.2. Troubleshooting the Connection

Even with the most meticulous attention to detail, something could still go wrong while setting up a terminal. Here is a list of common symptoms and some suggested fixes.

If no login prompt appears, make sure the terminal is plugged in and powered up. If it is a personal computer acting as a terminal, make sure it is running terminal emulation software on the correct serial port.

Make sure the cable is connected firmly to both the terminal and the FreeBSD computer. Make sure it is the right kind of cable.

Make sure the terminal and FreeBSD agree on the bps rate and parity settings. For a video display terminal, make sure the contrast and brightness controls are turned up. If it is a printing terminal, make sure paper and ink are in good supply.

Use `ps` to make sure that a `getty` process is running and serving the terminal. For example, the following listing shows that a `getty` is running on the second serial port, ttyu1, and is using the `std.38400` entry in /etc/gettytab:

```
# ps -axww|grep ttyu
22189  d1  Is+    0:00.03 /usr/libexec/getty std.38400 ttyu1
```

If no `getty` process is running, make sure the port is enabled in /etc/ttys. Remember to run `kill -HUP 1` after modifying /etc/ttys.

If the `getty` process is running but the terminal still does not display a login prompt, or if it displays a prompt but will not accept typed input, the terminal or cable may not support hardware handshaking. Try changing the entry in /etc/ttys from `std.38400` to `3wire.38400`, then run `kill -HUP 1` after modifying /etc/ttys. The `3wire` entry is similar to `std`, but ignores hardware handshaking. The bps may also need to be reduced or software flow control enabled when using `3wire` to prevent buffer overflows.

If garbage appears instead of a login prompt, make sure the terminal and FreeBSD agree on the bps rate and parity settings. Check the `getty` processes to make sure the correct *getty* type is in use. If not, edit /etc/ttys and run `kill -HUP 1`.

If characters appear doubled and the password appears when typed, switch the terminal, or the terminal emulation software, from "half duplex" or "local echo" to "full duplex."

29.4. Dial-in Service

Configuring a FreeBSD system for dial-in service is similar to configuring terminals, except that modems are used instead of terminal devices. FreeBSD supports both external and internal modems.

External modems are more convenient because they often can be configured via parameters stored in non-volatile RAM and they usually provide lighted indicators that display the state of important RS-232 signals, indicating whether the modem is operating properly.

Internal modems usually lack non-volatile RAM, so their configuration may be limited to setting DIP switches. If the internal modem has any signal indicator lights, they are difficult to view when the system's cover is in place.

When using an external modem, a proper cable is needed. A standard RS-232C serial cable should suffice.

FreeBSD needs the RTS and CTS signals for flow control at speeds above 2400 bps, the CD signal to detect when a call has been answered or the line has been hung up, and the DTR signal to reset the modem after a session is complete. Some cables are wired without all of the needed signals, so if a login session does not go away when the line hangs up, there may be a problem with the cable. Refer to Serial Cables and Ports for more information about these signals.

Like other UNIX®-like operating systems, FreeBSD uses the hardware signals to find out when a call has been answered or a line has been hung up and to hangup and reset the modem after a call. FreeBSD avoids sending commands to the modem or watching for status reports from the modem.

FreeBSD supports the NS8250, NS16450, NS16550, and NS16550A-based RS-232C (CCITT V.24) communications interfaces. The 8250 and 16450 devices have single-character buffers. The 16550 device provides a 16-character buffer, which allows for better system performance. Bugs in plain 16550 devices prevent the use of the 16-character buffer, so use 16550A devices if possible. As single-character-buffer devices require more work by the operating system than the 16-character-buffer devices, 16550A-based serial interface cards are preferred. If the system has many active serial ports or will have a heavy load, 16550A-based cards are better for low-error-rate communications.

The rest of this section demonstrates how to configure a modem to receive incoming connections, how to communicate with the modem, and offers some troubleshooting tips.

29.4.1. Modem Configuration

As with terminals, `init` spawns a `getty` process for each configured serial port used for dial-in connections. When a user dials the modem's line and the modems connect, the "Carrier Detect" signal is reported by the modem. The kernel notices that the carrier has been detected and instructs `getty` to open the port and display a `login:` prompt at the specified initial line speed. In a typical configuration, if garbage characters are received, usually due to the modem's connection speed being different than the configured speed, `getty` tries adjusting the line speeds until it receives reasonable characters. After the user enters their login name, `getty` executes `login`, which completes the login process by asking for the user's password and then starting the user's shell.

There are two schools of thought regarding dial-up modems. One configuration method is to set the modems and systems so that no matter at what speed a remote user dials in, the dial-in RS-232 interface runs at a locked speed. The benefit of this configuration is that the remote user always sees a system login prompt immediately. The downside is that the system does not know what a user's true data rate is, so full-screen programs like Emacs will not adjust their screen-painting methods to make their response better for slower connections.

The second method is to configure the RS-232 interface to vary its speed based on the remote user's connection speed. As `getty` does not understand any particular modem's connection speed reporting, it gives a `login:` message at an initial speed and watches the characters that come back in response. If the user sees junk, they should press `Enter` until they see a recognizable prompt. If the data rates do not match, `getty` sees anything the user types as junk, tries the next speed, and gives the `login:` prompt again. This procedure normally only takes a keystroke or two before the user sees a good prompt. This login sequence does not look as clean as the locked-speed method, but a user on a low-speed connection should receive better interactive response from full-screen programs.

When locking a modem's data communications rate at a particular speed, no changes to /etc/gettytab should be needed. However, for a matching-speed configuration, additional entries may be required in order to define the speeds to use for the modem. This example configures a 14.4 Kbps modem with a top interface speed of 19.2 Kbps using 8-bit, no parity connections. It configures `getty` to start the communications rate for a V.32bis connection at 19.2 Kbps, then cycles through 9600 bps, 2400 bps, 1200 bps, 300 bps, and back to 19.2 Kbps. Communications rate cycling is implemented with the `nx=` (next table) capability. Each line uses a `tc=` (table continuation) entry to pick up the rest of the settings for a particular data rate.

```
#
# Additions for a V.32bis Modem
#
um|V300|High Speed Modem at 300,8-bit:\
        :nx=V19200:tc=std.300:
un|V1200|High Speed Modem at 1200,8-bit:\
        :nx=V300:tc=std.1200:
uo|V2400|High Speed Modem at 2400,8-bit:\
        :nx=V1200:tc=std.2400:
up|V9600|High Speed Modem at 9600,8-bit:\
        :nx=V2400:tc=std.9600:
uq|V19200|High Speed Modem at 19200,8-bit:\
        :nx=V9600:tc=std.19200:
```

For a 28.8 Kbps modem, or to take advantage of compression on a 14.4 Kbps modem, use a higher communications rate, as seen in this example:

```
#
# Additions for a V.32bis or V.34 Modem
# Starting at 57.6 Kbps
#
vm|VH300|Very High Speed Modem at 300,8-bit:\
```

```
        :nx=VH57600:tc=std.300:
vn|VH1200|Very High Speed Modem at 1200,8-bit:\
        :nx=VH300:tc=std.1200:
vo|VH2400|Very High Speed Modem at 2400,8-bit:\
        :nx=VH1200:tc=std.2400:
vp|VH9600|Very High Speed Modem at 9600,8-bit:\
        :nx=VH2400:tc=std.9600:
vq|VH57600|Very High Speed Modem at 57600,8-bit:\
        :nx=VH9600:tc=std.57600:
```

For a slow CPU or a heavily loaded system without 16550A-based serial ports, this configuration may produce `uart` "silo" errors at 57.6 Kbps.

The configuration of /etc/ttys is similar to Configuring Terminal Entries, but a different argument is passed to `getty` and `dialup` is used for the terminal type. Replace *xxx* with the process `init` will run on the device:

```
ttyu0   "/usr/libexec/getty xxx"   dialup on
```

The `dialup` terminal type can be changed. For example, setting `vt102` as the default terminal type allows users to use VT102 emulation on their remote systems.

For a locked-speed configuration, specify the speed with a valid type listed in /etc/gettytab. This example is for a modem whose port speed is locked at 19.2 Kbps:

```
ttyu0   "/usr/libexec/getty std.19200"   dialup on
```

In a matching-speed configuration, the entry needs to reference the appropriate beginning "auto-baud" entry in /etc/gettytab. To continue the example for a matching-speed modem that starts at 19.2 Kbps, use this entry:

```
ttyu0   "/usr/libexec/getty V19200"   dialup on
```

After editing /etc/ttys, wait until the modem is properly configured and connected before signaling `init`:

```
# kill -HUP 1
```

High-speed modems, like V.32, V.32bis, and V.34 modems, use hardware (`RTS/CTS`) flow control. Use `stty` to set the hardware flow control flag for the modem port. This example sets the `crtscts` flag on COM2's dial-in and dial-out initialization devices:

```
# stty -f /dev/ttyu1.init crtscts
# stty -f /dev/cuau1.init crtscts
```

29.4.2. Troubleshooting

This section provides a few tips for troubleshooting a dial-up modem that will not connect to a FreeBSD system.

Hook up the modem to the FreeBSD system and boot the system. If the modem has status indication lights, watch to see whether the modem's DTR indicator lights when the `login:` prompt appears on the system's console. If it lights up, that should mean that FreeBSD has started a `getty` process on the appropriate communications port and is waiting for the modem to accept a call.

If the DTR indicator does not light, login to the FreeBSD system through the console and type `ps ax` to see if FreeBSD is running a `getty` process on the correct port:

```
  114 ??  I      0:00.10 /usr/libexec/getty V19200 ttyu0
```

If the second column contains a `d0` instead of a `??` and the modem has not accepted a call yet, this means that `getty` has completed its open on the communications port. This could indicate a problem with the cabling or a misconfigured modem because `getty` should not be able to open the communications port until the carrier detect signal has been asserted by the modem.

If no `getty` processes are waiting to open the port, double-check that the entry for the port is correct in /etc/ttys. Also, check /var/log/messages to see if there are any log messages from `init` or `getty`.

Next, try dialing into the system. Be sure to use 8 bits, no parity, and 1 stop bit on the remote system. If a prompt does not appear right away, or the prompt shows garbage, try pressing `Enter` about once per second. If there is still no `login:` prompt, try sending a `BREAK`. When using a high-speed modem, try dialing again after locking the dialing modem's interface speed.

If there is still no `login:` prompt, check /etc/gettytab again and double-check that:

- The initial capability name specified in the entry in /etc/ttys matches the name of a capability in /etc/gettytab.
- Each `nx=` entry matches another gettytab capability name.
- Each `tc=` entry matches another gettytab capability name.

If the modem on the FreeBSD system will not answer, make sure that the modem is configured to answer the phone when DTR is asserted. If the modem seems to be configured correctly, verify that the DTR line is asserted by checking the modem's indicator lights.

If it still does not work, try sending an email to the FreeBSD general questions mailing list describing the modem and the problem.

29.5. Dial-out Service

The following are tips for getting the host to connect over the modem to another computer. This is appropriate for establishing a terminal session with a remote host.

This kind of connection can be helpful to get a file on the Internet if there are problems using PPP. If

PPP is not working, use the terminal session to FTP the needed file. Then use zmodem to transfer it to the machine.

29.5.1. Using a Stock Hayes Modem

A generic Hayes dialer is built into `tip`. Use `at=hayes` in /etc/remote.

The Hayes driver is not smart enough to recognize some of the advanced features of newer modems messages like `BUSY`, `NO DIALTONE`, or `CONNECT 115200`. Turn those messages off when using `tip` with `ATX0&W`.

The dial timeout for `tip` is 60 seconds. The modem should use something less, or else `tip` will think there is a communication problem. Try `ATS7=45&W`.

29.5.2. Using `AT` Commands

Create a "direct" entry in /etc/remote. For example, if the modem is hooked up to the first serial port, /dev/cuau0, use the following line:

```
cuau0:dv=/dev/cuau0:br#19200:pa=none
```

Use the highest bps rate the modem supports in the `br` capability. Then, type `tip cuau0` to connect to the modem.

Or, use `cu` as `root` with the following command:

```
# cu -lline -sspeed
```

line is the serial port, such as /dev/cuau0, and *speed* is the speed, such as `57600`. When finished entering the AT commands, type `~.` to exit.

29.5.3. The `@` Sign Does Not Work

The `@` sign in the phone number capability tells `tip` to look in /etc/phones for a phone number. But, the `@` sign is also a special character in capability files like /etc/remote, so it needs to be escaped with a backslash:

```
pn=\@
```

29.5.4. Dialing from the Command Line

Put a "generic" entry in /etc/remote. For example:

```
tip115200|Dial any phone number at 115200 bps:\
        :dv=/dev/cuau0:br#115200:at=hayes:pa=none:du:
tip57600|Dial any phone number at 57600 bps:\
```

```
        :dv=/dev/cuau0:br#57600:at=hayes:pa=none:du:
```

This should now work:

```
# tip -115200 5551234
```

Users who prefer `cu` over `tip`, can use a generic `cu` entry:

```
cu115200|Use cu to dial any number at 115200bps:\
        :dv=/dev/cuau1:br#57600:at=hayes:pa=none:du:
```

and type:

```
# cu 5551234 -s 115200
```

29.5.5. Setting the bps Rate

Put in an entry for `tip1200` or `cu1200`, but go ahead and use whatever bps rate is appropriate with the `br` capability. `tip` thinks a good default is 1200 bps which is why it looks for a `tip1200` entry. 1200 bps does not have to be used, though.

29.5.6. Accessing a Number of Hosts Through a Terminal Server

Rather than waiting until connected and typing `CONNECT host` each time, use `tip`'s `cm` capability. For example, these entries in /etc/remote allow typing `tip pain` or `tip muffin` to connect to the hosts `pain` or `muffin`, and `tip deep13` to connect to the terminal server.

```
pain|pain.deep13.com|Forrester's machine:\
        :cm=CONNECT pain\n:tc=deep13:
muffin|muffin.deep13.com|Frank's machine:\
        :cm=CONNECT muffin\n:tc=deep13:
deep13:Gizmonics Institute terminal server:\
        :dv=/dev/cuau2:br#38400:at=hayes:du:pa=none:pn=5551234:
```

29.5.7. Using More Than One Line with `tip`

This is often a problem where a university has several modem lines and several thousand students trying to use them.

Make an entry in /etc/remote and use `@` for the `pn` capability:

```
big-university:\
        :pn=\@:tc=dialout
dialout:\
```

```
        :dv=/dev/cuau3:br#9600:at=courier:du:pa=none:
```

Then, list the phone numbers in /etc/phones:

```
big-university 5551111
big-university 5551112
big-university 5551113
big-university 5551114
```

`tip` will try each number in the listed order, then give up. To keep retrying, run `tip` in a `while` loop.

29.5.8. Using the Force Character

`Ctrl` + `P` is the default "force" character, used to tell `tip` that the next character is literal data. The force character can be set to any other character with the `~s` escape, which means "set a variable."

Type `~sforce=single-char` followed by a newline. *single-char* is any single character. If *single-char* is left out, then the force character is the null character, which is accessed by typing `Ctrl` + `2` or `Ctrl` + `Space`. A pretty good value for *single-char* is `Shift` + `Ctrl` + `6`, which is only used on some terminal servers.

To change the force character, specify the following in ~/.tiprc:

```
force=single-char
```

29.5.9. Upper Case Characters

This happens when `Ctrl` + `A` is pressed, which is `tip`'s "raise character", specially designed for people with broken caps-lock keys. Use `~s` to set `raisechar` to something reasonable. It can be set to be the same as the force character, if neither feature is used.

Here is a sample ~/.tiprc for Emacs users who need to type `Ctrl` + `2` and `Ctrl` + `A`:

```
force=^^
raisechar=^^
```

The `^^` is `Shift` + `Ctrl` + `6`.

29.5.10. File Transfers with `tip`

When talking to another UNIX®-like operating system, files can be sent and received using `~p` (put) and `~t` (take). These commands run `cat` and `echo` on the remote system to accept and send files. The syntax is: `~p` local-file [remote-file] `~t` remote-file [local-file]

There is no error checking, so another protocol, like zmodem, should probably be used.

29.5.11. Using zmodem with `tip`?

To receive files, start the sending program on the remote end. Then, type `~C rz` to begin receiving them locally.

To send files, start the receiving program on the remote end. Then, type `~C sz files` to send them to the remote system.

29.6. Setting Up the Serial Console

FreeBSD has the ability to boot a system with a dumb terminal on a serial port as a console. This configuration is useful for system administrators who wish to install FreeBSD on machines that have no keyboard or monitor attached, and developers who want to debug the kernel or device drivers.

As described in The FreeBSD Booting Process, FreeBSD employs a three stage bootstrap. The first two stages are in the boot block code which is stored at the beginning of the FreeBSD slice on the boot disk. The boot block then loads and runs the boot loader as the third stage code.

In order to set up booting from a serial console, the boot block code, the boot loader code, and the kernel need to be configured.

29.6.1. Quick Serial Console Configuration

This section provides a fast overview of setting up the serial console. This procedure can be used when the dumb terminal is connected to COM1.

Procedure: Configuring a Serial Console on COM1

1. Connect the serial cable to COM1 and the controlling terminal.
2. To configure boot messages to display on the serial console, issue the following command as the superuser:

```
# echo 'console="comconsole"' >> /boot/loader.conf
```

3. Edit /etc/ttys and change `off` to `on` and `dialup` to `vt100` for the ttyu0 entry. Otherwise, a password will not be required to connect via the serial console, resulting in a potential security hole.
4. Reboot the system to see if the changes took effect.

If a different configuration is required, see the next section for a more in-depth configuration explanation.

29.6.2. In-Depth Serial Console Configuration

This section provides a more detailed explanation of the steps needed to setup a serial console in FreeBSD.

Procedure: Configuring a Serial Console

1. Prepare a serial cable.

   Use either a null-modem cable or a standard serial cable and a null-modem adapter. See Serial Cables and Ports for a discussion on serial cables.

2. Unplug the keyboard.

   Many systems probe for the keyboard during the Power-On Self-Test (POST) and will generate an error if the keyboard is not detected. Some machines will refuse to boot until the keyboard is plugged in.

   If the computer complains about the error, but boots anyway, no further configuration is needed.

   If the computer refuses to boot without a keyboard attached, configure the BIOS so that it ignores this error. Consult the motherboard's manual for details on how to do this.

   Try setting the keyboard to "Not installed" in the BIOS. This setting tells the BIOS not to probe for a keyboard at power-on so it should not complain if the keyboard is absent. If that option is not present in the BIOS, look for an "Halt on Error" option instead. Setting this to "All but Keyboard" or to "No Errors" will have the same effect.

   If the system has a PS/2® mouse, unplug it as well. PS/2® mice share some hardware with the keyboard and leaving the mouse plugged in can fool the keyboard probe into thinking the keyboard is still there.

   While most systems will boot without a keyboard, quite a few will not boot without a graphics adapter. Some systems can be configured to boot with no graphics adapter by changing the "graphics adapter" setting in the BIOS configuration to "Not installed". Other systems do not support this option and will refuse to boot if there is no display hardware in the system. With these machines, leave some kind of graphics card plugged in, even if it is just a junky mono board. A monitor does not need to be attached.

3. Plug a dumb terminal, an old computer with a modem program, or the serial port on another UNIX® box into the serial port.
4. Add the appropriate `hint.uart.*` entries to /boot/device.hints for the serial port. Some multi-port cards also require kernel configuration options. Refer to uart(4) for the required options and device hints for each supported serial port.
5. Create boot.config in the root directory of the `a` partition on the boot drive.

   This file instructs the boot block code how to boot the system. In order to activate the serial console, one or more of the following options are needed. When using multiple options, include them all on the same line:

   `-h`

   Toggles between the internal and serial consoles. Use this to switch console devices. For

instance, to boot from the internal (video) console, use `-h` to direct the boot loader and the kernel to use the serial port as its console device. Alternatively, to boot from the serial port, use `-h` to tell the boot loader and the kernel to use the video display as the console instead.

`-D`

Toggles between the single and dual console configurations. In the single configuration, the console will be either the internal console (video display) or the serial port, depending on the state of `-h`. In the dual console configuration, both the video display and the serial port will become the console at the same time, regardless of the state of `-h`. However, the dual console configuration takes effect only while the boot block is running. Once the boot loader gets control, the console specified by `-h` becomes the only console.

`-P`

Makes the boot block probe the keyboard. If no keyboard is found, the `-D` and `-h` options are automatically set.



Due to space constraints in the current version of the boot blocks, `-P` is capable of detecting extended keyboards only. Keyboards with less than 101 keys and without F11 and F12 keys may not be detected. Keyboards on some laptops may not be properly found because of this limitation. If this is the case, do not use `-P`.

Use either `-P` to select the console automatically or `-h` to activate the serial console. Refer to boot(8) and boot.config(5) for more details.

The options, except for `-P`, are passed to the boot loader. The boot loader will determine whether the internal video or the serial port should become the console by examining the state of `-h`. This means that if `-D` is specified but `-h` is not specified in /boot.config, the serial port can be used as the console only during the boot block as the boot loader will use the internal video display as the console.

6. Boot the machine.

When FreeBSD starts, the boot blocks echo the contents of /boot.config to the console. For example:

```
/boot.config: -P
Keyboard: no
```

The second line appears only if `-P` is in /boot.config and indicates the presence or absence of the keyboard. These messages go to either the serial or internal console, or both, depending on the option in /boot.config:

Options	Message goes to
none	internal console
`-h`	serial console

Options	Message goes to
`-D`	serial and internal consoles
`-Dh`	serial and internal consoles
`-P`, keyboard present	internal console
`-P`, keyboard absent	serial console

After the message, there will be a small pause before the boot blocks continue loading the boot loader and before any further messages are printed to the console. Under normal circumstances, there is no need to interrupt the boot blocks, but one can do so in order to make sure things are set up correctly.

Press any key, other than `Enter`, at the console to interrupt the boot process. The boot blocks will then prompt for further action:

```
>> FreeBSD/i386 BOOT
Default: 0:ad(0,a)/boot/loader
boot:
```

Verify that the above message appears on either the serial or internal console, or both, according to the options in /boot.config. If the message appears in the correct console, press `Enter` to continue the boot process.

If there is no prompt on the serial terminal, something is wrong with the settings. Enter `-h` then `Enter` or `Return` to tell the boot block (and then the boot loader and the kernel) to choose the serial port for the console. Once the system is up, go back and check what went wrong.

During the third stage of the boot process, one can still switch between the internal console and the serial console by setting appropriate environment variables in the boot loader. See loader(8) for more information.



This line in /boot/loader.conf or /boot/loader.conf.local configures the boot loader and the kernel to send their boot messages to the serial console, regardless of the options in /boot.config:

```
console="comconsole"
```

That line should be the first line of /boot/loader.conf so that boot messages are displayed on the serial console as early as possible.

If that line does not exist, or if it is set to `console="vidconsole"`, the boot loader and the kernel will use whichever console is indicated by `-h` in the boot block. See loader.conf(5) for more information.

At the moment, the boot loader has no option equivalent to `-P` in the boot block, and there is no provision to automatically select the internal console and the serial

console based on the presence of the keyboard.



While it is not required, it is possible to provide a `login` prompt over the serial line. To configure this, edit the entry for the serial port in /etc/ttys using the instructions in Terminal Configuration. If the speed of the serial port has been changed, change `std.115200` to match the new setting.

29.6.3. Setting a Faster Serial Port Speed

By default, the serial port settings are 115200 baud, 8 bits, no parity, and 1 stop bit. To change the default console speed, use one of the following options:

- Edit /etc/make.conf and set `BOOT_COMCONSOLE_SPEED` to the new console speed. Then, recompile and install the boot blocks and the boot loader:

```
# cd /sys/boot
# make clean
# make
# make install
```

  If the serial console is configured in some other way than by booting with `-h`, or if the serial console used by the kernel is different from the one used by the boot blocks, add the following option, with the desired speed, to a custom kernel configuration file and compile a new kernel:

```
options CONSPEED=19200
```

- Add the `-S19200` boot option to /boot.config, replacing `19200` with the speed to use.
- Add the following options to /boot/loader.conf. Replace `115200` with the speed to use.

```
boot_multicons="YES"
boot_serial="YES"
comconsole_speed="115200"
console="comconsole,vidconsole"
```

29.6.4. Entering the DDB Debugger from the Serial Line

To configure the ability to drop into the kernel debugger from the serial console, add the following options to a custom kernel configuration file and compile the kernel using the instructions in Configuring the FreeBSD Kernel. Note that while this is useful for remote diagnostics, it is also dangerous if a spurious BREAK is generated on the serial port. Refer to ddb(4) and ddb(8) for more information about the kernel debugger.

```
options BREAK_TO_DEBUGGER
options DDB
```

Chapter 30. PPP

30.1. Synopsis

FreeBSD supports the Point-to-Point (PPP) protocol which can be used to establish a network or Internet connection using a dial-up modem. This chapter describes how to configure modem-based communication services in FreeBSD.

Read this chapter to learn:

- How to configure, use, and troubleshoot a PPP connection.
- How to set up PPP over Ethernet (PPPoE).
- How to set up PPP over ATM (PPPoA).

Before reading this chapter:

- Be familiar with basic network terminology.
- Understand the basics and purpose of a dial-up connection and PPP.

30.2. Configuring PPP

FreeBSD provides built-in support for managing dial-up PPP connections using ppp(8). The default FreeBSD kernel provides support for tun which is used to interact with a modem hardware. Configuration is performed by editing at least one configuration file, and configuration files containing examples are provided. Finally, `ppp` is used to start and manage connections.

In order to use a PPP connection, the following items are needed:

- A dial-up account with an Internet Service Provider (ISP).
- A dial-up modem.
- The dial-up number for the ISP.
- The login name and password assigned by the ISP.
- The IP address of one or more DNS servers. Normally, the ISP provides these addresses. If it did not, FreeBSD can be configured to use DNS negotiation.

If any of the required information is missing, contact the ISP.

The following information may be supplied by the ISP, but is not necessary:

- The IP address of the default gateway. If this information is unknown, the ISP will automatically provide the correct value during connection setup. When configuring PPP on FreeBSD, this address is referred to as `HISADDR`.
- The subnet mask. If the ISP has not provided one, `255.255.255.255` will be used in the ppp(8) configuration file. *

If the ISP has assigned a static IP address and hostname, it should be input into the configuration file. Otherwise, this information will be automatically provided during connection setup.

The rest of this section demonstrates how to configure FreeBSD for common PPP connection scenarios. The required configuration file is /etc/ppp/ppp.conf and additional files and examples are available in /usr/share/examples/ppp/.



Throughout this section, many of the file examples display line numbers. These line numbers have been added to make it easier to follow the discussion and are not meant to be placed in the actual file.

When editing a configuration file, proper indentation is important. Lines that end in a `:` start in the first column (beginning of the line) while all other lines should be indented as shown using spaces or tabs.

30.2.1. Basic Configuration

In order to configure a PPP connection, first edit /etc/ppp/ppp.conf with the dial-in information for the ISP. This file is described as follows:

```
1     default:
2       set log Phase Chat LCP IPCP CCP tun command
3       ident user-ppp VERSION
4       set device /dev/cuau0
5       set speed 115200
6       set dial "ABORT BUSY ABORT NO\\sCARRIER TIMEOUT 5 \
7                 \"\" AT OK-AT-OK ATE1Q0 OK \\dATDT\\T TIMEOUT 40 CONNECT"
8       set timeout 180
9       enable dns
10
11    provider:
12      set phone "(123) 456 7890"
13      set authname foo
14      set authkey bar
15      set timeout 300
16      set ifaddr x.x.x.x/0 y.y.y.y/0 255.255.255.255 0.0.0.0
17      add default HISADDR
```

Line 1

Identifies the `default` entry. Commands in this entry (lines 2 through 9) are executed automatically when `ppp` is run.

Line 2

Enables verbose logging parameters for testing the connection. Once the configuration is working satisfactorily, this line should be reduced to:

```
set log phase tun
```

Line 3

Displays the version of ppp(8) to the PPP software running on the other side of the connection.

Line 4

Identifies the device to which the modem is connected, where COM1 is /dev/cuau0 and COM2 is /dev/cuau1.

Line 5

Sets the connection speed. If `115200` does not work on an older modem, try `38400` instead.

Lines 6 & 7

The dial string written as an expect-send syntax. Refer to chat(8) for more information.

Note that this command continues onto the next line for readability. Any command in ppp.conf may do this if the last character on the line is `\`.

Line 8

Sets the idle timeout for the link in seconds.

Line 9

Instructs the peer to confirm the DNS settings. If the local network is running its own DNS server, this line should be commented out, by adding a `#` at the beginning of the line, or removed.

Line 10

A blank line for readability. Blank lines are ignored by ppp(8).

Line 11

Identifies an entry called `provider`. This could be changed to the name of the ISP so that `load ISP` can be used to start the connection.

Line 12

Use the phone number for the ISP. Multiple phone numbers may be specified using the colon (`:`) or pipe character (`|`) as a separator. To rotate through the numbers, use a colon. To always attempt to dial the first number first and only use the other numbers if the first number fails, use the pipe character. Always enclose the entire set of phone numbers between quotation marks (`"`) to prevent dialing failures.

Lines 13 & 14

Use the user name and password for the ISP.

Line 15

Sets the default idle timeout in seconds for the connection. In this example, the connection will be closed automatically after 300 seconds of inactivity. To prevent a timeout, set this value to zero.

Line 16

Sets the interface addresses. The values used depend upon whether a static IP address has been obtained from the ISP or if it instead negotiates a dynamic IP address during connection.

If the ISP has allocated a static IP address and default gateway, replace *x.x.x.x* with the static IP address and replace *y.y.y.y* with the IP address of the default gateway. If the ISP has only provided a static IP address without a gateway address, replace *y.y.y.y* with `10.0.0.2/0`.

If the IP address changes whenever a connection is made, change this line to the following value. This tells ppp(8) to use the IP Configuration Protocol (IPCP) to negotiate a dynamic IP address:

```
set ifaddr 10.0.0.1/0 10.0.0.2/0 255.255.255.255 0.0.0.0
```

Line 17

Keep this line as-is as it adds a default route to the gateway. The `HISADDR` will automatically be replaced with the gateway address specified on line 16. It is important that this line appears after line 16.

Depending upon whether ppp(8) is started manually or automatically, a /etc/ppp/ppp.linkup may also need to be created which contains the following lines. This file is required when running `ppp` in `-auto` mode. This file is used after the connection has been established. At this point, the IP address will have been assigned and it is now possible to add the routing table entries. When creating this file, make sure that *provider* matches the value demonstrated in line 11 of ppp.conf.

```
provider:
      add default HISADDR
```

This file is also needed when the default gateway address is "guessed" in a static IP address configuration. In this case, remove line 17 from ppp.conf and create /etc/ppp/ppp.linkup with the above two lines. More examples for this file can be found in /usr/share/examples/ppp/.

By default, `ppp` must be run as `root`. To change this default, add the account of the user who should run `ppp` to the `network` group in /etc/group.

Then, give the user access to one or more entries in /etc/ppp/ppp.conf with `allow`. For example, to give `fred` and `mary` permission to only the `provider:` entry, add this line to the `provider:` section:

```
allow users fred mary
```

To give the specified users access to all entries, put that line in the `default` section instead.

30.2.2. Advanced Configuration

It is possible to configure PPP to supply DNS and NetBIOS nameserver addresses on demand.

To enable these extensions with PPP version 1.x, the following lines might be added to the relevant

section of /etc/ppp/ppp.conf.

```
enable msext
set ns 203.14.100.1 203.14.100.2
set nbns 203.14.100.5
```

And for PPP version 2 and above:

```
accept dns
set dns 203.14.100.1 203.14.100.2
set nbns 203.14.100.5
```

This will tell the clients the primary and secondary name server addresses, and a NetBIOS nameserver host.

In version 2 and above, if the `set dns` line is omitted, PPP will use the values found in /etc/resolv.conf.

30.2.2.1. PAP and CHAP Authentication

Some ISPs set their system up so that the authentication part of the connection is done using either of the PAP or CHAP authentication mechanisms. If this is the case, the ISP will not give a `login:` prompt at connection, but will start talking PPP immediately.

PAP is less secure than CHAP, but security is not normally an issue here as passwords, although being sent as plain text with PAP, are being transmitted down a serial line only. There is not much room for crackers to "eavesdrop".

The following alterations must be made:

```
13      set authname MyUserName
14      set authkey MyPassword
15      set login
```

Line 13

This line specifies the PAP/CHAP user name.Insert the correct value for *MyUserName*.

Line 14

This line specifies the PAP/CHAP password. Insert the correct value for *MyPassword*. An additional line may be added, such as:

```
16      accept PAP
```

or

```
16      accept CHAP
```

to make it obvious that this is the intention, but PAP and CHAP are both accepted by default.

Line 15

The ISP will not normally require a login to the server when using PAP or CHAP. Therefore, disable the "set login" string.

30.2.2.2. Using PPP Network Address Translation Capability

PPP has ability to use internal NAT without kernel diverting capabilities. This functionality may be enabled by the following line in /etc/ppp/ppp.conf:

```
nat enable yes
```

Alternatively, NAT may be enabled by command-line option `-nat`. There is also /etc/rc.conf knob named `ppp_nat`, which is enabled by default.

When using this feature, it may be useful to include the following /etc/ppp/ppp.conf options to enable incoming connections forwarding:

```
nat port tcp 10.0.0.2:ftp ftp
nat port tcp 10.0.0.2:http http
```

or do not trust the outside at all

```
nat deny_incoming yes
```

30.2.3. Final System Configuration

While `ppp` is now configured, some edits still need to be made to /etc/rc.conf.

Working from the top down in this file, make sure the `hostname=` line is set:

```
hostname="foo.example.com"
```

If the ISP has supplied a static IP address and name, use this name as the host name.

Look for the `network_interfaces` variable. To configure the system to dial the ISP on demand, make sure the tun0 device is added to the list, otherwise remove it.

```
network_interfaces="lo0 tun0"
ifconfig_tun0=
```



The ifconfig_tun0 variable should be empty, and a file called /etc/start_if.tun0 should be created. This file should contain the line:

```
ppp -auto mysystem
```

This script is executed at network configuration time, starting the ppp daemon in automatic mode. If this machine acts as a gateway, consider including -alias. Refer to the manual page for further details.

Make sure that the router program is set to NO with the following line in /etc/rc.conf:

```
router_enable="NO"
```

It is important that the routed daemon is not started, as routed tends to delete the default routing table entries created by ppp.

It is probably a good idea to ensure that the sendmail_flags line does not include the -q option, otherwise sendmail will attempt to do a network lookup every now and then, possibly causing the machine to dial out. Try this:

```
sendmail_flags="-bd"
```

The downside is that sendmail is forced to re-examine the mail queue whenever the ppp link. To automate this, include !bg in ppp.linkup:

```
1     provider:
2       delete ALL
3       add 0 0 HISADDR
4       !bg sendmail -bd -q30m
```

An alternative is to set up a "dfilter" to block SMTP traffic. Refer to the sample files for further details.

30.2.4. Using ppp

All that is left is to reboot the machine. After rebooting, either type:

```
# ppp
```

and then dial provider to start the PPP session, or, to configure ppp to establish sessions automatically when there is outbound traffic and start_if.tun0 does not exist, type:

```
# ppp -auto provider
```

It is possible to talk to the ppp program while it is running in the background, but only if a suitable diagnostic port has been set up. To do this, add the following line to the configuration:

```
set server /var/run/ppp-tun%d DiagnosticPassword 0177
```

This will tell PPP to listen to the specified UNIX® domain socket, asking clients for the specified password before allowing access. The %d in the name is replaced with the tun device number that is in use.

Once a socket has been set up, the pppctl(8) program may be used in scripts that wish to manipulate the running program.

30.2.5. Configuring Dial-in Services

"Dial-in Service" provides a good description on enabling dial-up services using getty(8).

An alternative to getty is comms/mgetty+sendfax port), a smarter version of getty designed with dial-up lines in mind.

The advantages of using mgetty is that it actively *talks* to modems, meaning if port is turned off in /etc/ttys then the modem will not answer the phone.

Later versions of mgetty (from 0.99beta onwards) also support the automatic detection of PPP streams, allowing clients scriptless access to the server.

Refer to http://mgetty.greenie.net/doc/mgetty_toc.html for more information on mgetty.

By default the comms/mgetty+sendfax port comes with the AUTO_PPP option enabled allowing mgetty to detect the LCP phase of PPP connections and automatically spawn off a ppp shell. However, since the default login/password sequence does not occur it is necessary to authenticate users using either PAP or CHAP.

This section assumes the user has successfully compiled, and installed the comms/mgetty+sendfax port on his system.

Ensure that /usr/local/etc/mgetty+sendfax/login.config has the following:

```
/AutoPPP/ -     - /etc/ppp/ppp-pap-dialup
```

This tells mgetty to run ppp-pap-dialup for detected PPP connections.

Create an executable file called /etc/ppp/ppp-pap-dialup containing the following:

```
#!/bin/sh
exec /usr/sbin/ppp -direct pap$IDENT
```

For each dial-up line enabled in /etc/ttys, create a corresponding entry in /etc/ppp/ppp.conf. This

will happily co-exist with the definitions we created above.

```
pap:
  enable pap
  set ifaddr 203.14.100.1 203.14.100.20-203.14.100.40
  enable proxy
```

Each user logging in with this method will need to have a username/password in /etc/ppp/ppp.secret, or alternatively add the following option to authenticate users via PAP from /etc/passwd.

```
enable passwdauth
```

To assign some users a static IP number, specify the number as the third argument in /etc/ppp/ppp.secret. See /usr/share/examples/ppp/ppp.secret.sample for examples.

30.3. Troubleshooting PPP Connections

This section covers a few issues which may arise when using PPP over a modem connection. Some ISPs present the `ssword` prompt while others present `password`. If the `ppp` script is not written accordingly, the login attempt will fail. The most common way to debug `ppp` connections is by connecting manually as described in this section.

30.3.1. Check the Device Nodes

When using a custom kernel, make sure to include the following line in the kernel configuration file:

```
device   uart
```

The uart device is already included in the `GENERIC` kernel, so no additional steps are necessary in this case. Just check the `dmesg` output for the modem device with:

```
# dmesg | grep uart
```

This should display some pertinent output about the uart devices. These are the COM ports we need. If the modem acts like a standard serial port, it should be listed on uart1, or COM2. If so, a kernel rebuild is not required. When matching up, if the modem is on uart1, the modem device would be /dev/cuau1.

30.3.2. Connecting Manually

Connecting to the Internet by manually controlling `ppp` is quick, easy, and a great way to debug a connection or just get information on how the ISP treats `ppp` client connections. Lets start PPP from

the command line. Note that in all of our examples we will use *example* as the hostname of the machine running PPP. To start `ppp`:

```
# ppp
```

```
ppp ON example> set device /dev/cuau1
```

This second command sets the modem device to cuau1.

```
ppp ON example> set speed 115200
```

This sets the connection speed to 115,200 kbps.

```
ppp ON example> enable dns
```

This tells `ppp` to configure the resolver and add the nameserver lines to /etc/resolv.conf. If `ppp` cannot determine the hostname, it can manually be set later.

```
ppp ON example> term
```

This switches to "terminal" mode in order to manually control the modem.

```
deflink: Entering terminal mode on /dev/cuau1
type '~h' for help
```

```
at
OK
atdt123456789
```

Use `at` to initialize the modem, then use `atdt` and the number for the ISP to begin the dial in process.

```
CONNECT
```

Confirmation of the connection, if we are going to have any connection problems, unrelated to hardware, here is where we will attempt to resolve them.

```
ISP Login:myusername
```

At this prompt, return the prompt with the username that was provided by the ISP.

```
ISP Pass:mypassword
```

At this prompt, reply with the password that was provided by the ISP. Just like logging into FreeBSD, the password will not echo.

```
Shell or PPP:ppp
```

Depending on the ISP, this prompt might not appear. If it does, it is asking whether to use a shell on the provider or to start `ppp`. In this example, `ppp` was selected in order to establish an Internet connection.

```
Ppp ON example>
```

Notice that in this example the first `p` has been capitalized. This shows that we have successfully connected to the ISP.

```
Ppp ON example>
```

We have successfully authenticated with our ISP and are waiting for the assigned IP address.

```
PPP ON example>
```

We have made an agreement on an IP address and successfully completed our connection.

```
PPP ON example>add default HISADDR
```

Here we add our default route, we need to do this before we can talk to the outside world as currently the only established connection is with the peer. If this fails due to existing routes, put a bang character `!` in front of the `add`. Alternatively, set this before making the actual connection and it will negotiate a new route accordingly.

If everything went good we should now have an active connection to the Internet, which could be thrown into the background using `CTRL` + `z`. If `PPP` returns to `ppp` the connection has been lost. This is good to know because it shows the connection status. Capital P's represent a connection to the ISP and lowercase p's show that the connection has been lost.

30.3.3. Debugging

If a connection cannot be established, turn hardware flow CTS/RTS to off using `set ctsrts off`. This is mainly the case when connected to some PPP-capable terminal servers, where PPP hangs when it tries to write data to the communication link, and waits for a Clear To Send (CTS) signal which may never come. When using this option, include `set accmap` as it may be required to defeat hardware dependent on passing certain characters from end to end, most of the time XON/XOFF. Refer to

ppp(8) for more information on this option and how it is used.

An older modem may need `set parity even`. Parity is set at none be default, but is used for error checking with a large increase in traffic, on older modems.

PPP may not return to the command mode, which is usually a negotiation error where the ISP is waiting for negotiating to begin. At this point, using `~p` will force ppp to start sending the configuration information.

If a login prompt never appears, PAP or CHAP authentication is most likely required. To use PAP or CHAP, add the following options to PPP before going into terminal mode:

```
ppp ON example> set authname myusername
```

Where *myusername* should be replaced with the username that was assigned by the ISP.

```
ppp ON example> set authkey mypassword
```

Where *mypassword* should be replaced with the password that was assigned by the ISP.

If a connection is established, but cannot seem to find any domain name, try to ping(8) an IP address. If there is 100 percent (100%) packet loss, it is likely that a default route was not assigned. Double check that `add default HISADDR` was set during the connection. If a connection can be made to a remote IP address, it is possible that a resolver address has not been added to /etc/resolv.conf. This file should look like:

```
domain example.com
nameserver x.x.x.x
nameserver y.y.y.y
```

Where *x.x.x.x* and *y.y.y.y* should be replaced with the IP address of the ISP's DNS servers.

To configure syslog(3) to provide logging for the PPP connection, make sure this line exists in /etc/syslog.conf:

```
!ppp
*.*     /var/log/ppp.log
```

30.4. Using PPP over Ethernet (PPPoE)

This section describes how to set up PPP over Ethernet (PPPoE).

Here is an example of a working ppp.conf:

```
default:
```

```
  set log Phase tun command # add more detailed logging when needed
  set ifaddr 10.0.0.1/0 10.0.0.2/0

name_of_service_provider:
  set device PPPoE:xl1 # replace xl1 with the Ethernet device
  set authname THELOGINNAME
  set authkey THEPASSWORD
  set dial
  set login
  add default HISADDR
```

As `root`, run:

```
# ppp -ddial name_of_service_provider
```

Add the following to /etc/rc.conf:

```
ppp_enable="YES"
ppp_mode="ddial"
ppp_nat="YES"   # when needing to enable nat for the local network, otherwise NO
ppp_profile="name_of_service_provider"
```

30.4.1. Using a PPPoE Service Tag

Sometimes it will be necessary to use a service tag to establish the connection. Service tags are used to distinguish between different PPPoE servers attached to a given network.

Any required service tag information should be in the documentation provided by the ISP.

As a last resort, one could try installing the net/rr-pppoe package or port. Bear in mind however, this may de-program the modem and render it useless, so think twice before doing it. Simply install the program shipped with the modem. Then, access the **System** menu from the program. The name of the profile should be listed there. It is usually *ISP*.

The profile name (service tag) will be used in the PPPoE configuration entry in ppp.conf as the provider part for `set device`. Refer to ppp(8) for full details. It should look like this:

```
set device PPPoE:xl1:ISP
```

Do not forget to change *xl1* to the proper device for the Ethernet card.

Do not forget to change *ISP* to the profile.

For additional information, refer to Cheaper Broadband with FreeBSD on DSL by Renaud Waldura.

30.4.2. PPPoE with a 3Com® HomeConnect™ ADSL Modem Dual Link

This modem does not follow the PPPoE specification defined in RFC 2516.

In order to make FreeBSD capable of communicating with this device, a sysctl must be set. This can be done automatically at boot time by updating /etc/sysctl.conf:

```
net.graph.nonstandard_pppoe=1
```

or can be done immediately with the command:

```
# sysctl net.graph.nonstandard_pppoe=1
```

Unfortunately, because this is a system-wide setting, it is not possible to talk to a normal PPPoE client or server and a 3Com® HomeConnect™ ADSL Modem at the same time.

30.5. Using PPP over ATM (PPPoA)

The following describes how to set up PPP over ATM (PPPoA). PPPoA is a popular choice among European DSL providers.

30.5.1. Using mpd

The mpd application can be used to connect to a variety of services, in particular PPTP services. It can be installed using the net/mpd5 package or port. Many ADSL modems require that a PPTP tunnel is created between the modem and computer.

Once installed, configure mpd to suit the provider's settings. The port places a set of sample configuration files which are well documented in /usr/local/etc/mpd/. A complete guide to configure mpd is available in HTML format in /usr/ports/shared/doc/mpd/. Here is a sample configuration for connecting to an ADSL service with mpd. The configuration is spread over two files, first the mpd.conf:



This example mpd.conf only works with mpd 4.x.

```
default:
    load adsl

adsl:
    new -i ng0 adsl adsl
    set bundle authname username ①
    set bundle password password ②
    set bundle disable multilink

    set link no pap acfcomp protocomp
    set link disable chap
```

```
    set link accept chap
    set link keep-alive 30 10

    set ipcp no vjcomp
    set ipcp ranges 0.0.0.0/0 0.0.0.0/0

    set iface route default
    set iface disable on-demand
    set iface enable proxy-arp
    set iface idle 0

    open
```

① The username used to authenticate with the ISP.

② The password used to authenticate with the ISP.

Information about the link, or links, to establish is found in mpd.links. An example mpd.links to accompany the above example is given beneath:

```
adsl:
    set link type pptp
    set pptp mode active
    set pptp enable originate outcall
    set pptp self 10.0.0.1 ①
    set pptp peer 10.0.0.138 ②
```

① The IP address of FreeBSD computer running mpd.

② The IP address of the ADSL modem. The Alcatel SpeedTouch™ Home defaults to `10.0.0.138`.

It is possible to initialize the connection easily by issuing the following command as `root`:

```
# mpd -b adsl
```

To view the status of the connection:

```
% ifconfig ng0
ng0: flags=88d1<UP,POINTOPOINT,RUNNING,NOARP,SIMPLEX,MULTICAST> mtu 1500
     inet 216.136.204.117 --> 204.152.186.171 netmask 0xffffffff
```

Using mpd is the recommended way to connect to an ADSL service with FreeBSD.

30.5.2. Using pptpclient

It is also possible to use FreeBSD to connect to other PPPoA services using net/pptpclient.

To use net/pptpclient to connect to a DSL service, install the port or package, then edit /etc/ppp/ppp.conf. An example section of ppp.conf is given below. For further information on

ppp.conf options consult ppp(8).

```
adsl:
 set log phase chat lcp ipcp ccp tun command
 set timeout 0
 enable dns
 set authname username ①
 set authkey password ②
 set ifaddr 0 0
 add default HISADDR
```

① The username for the DSL provider.

② The password for the account.



Since the account's password is added to ppp.conf in plain text form, make sure nobody can read the contents of this file:

```
# chown root:wheel /etc/ppp/ppp.conf
# chmod 600 /etc/ppp/ppp.conf
```

This will open a tunnel for a PPP session to the DSL router. Ethernet DSL modems have a preconfigured LAN IP address to connect to. In the case of the Alcatel SpeedTouch™ Home, this address is 10.0.0.138. The router's documentation should list the address the device uses. To open the tunnel and start a PPP session:

```
# pptp address adsl
```



If an ampersand ("&") is added to the end of this command, pptp will return the prompt.

A tun virtual tunnel device will be created for interaction between the pptp and ppp processes. Once the prompt is returned, or the pptp process has confirmed a connection, examine the tunnel:

```
% ifconfig tun0
tun0: flags=8051<UP,POINTOPOINT,RUNNING,MULTICAST> mtu 1500
        inet 216.136.204.21 --> 204.152.186.171 netmask 0xffffff00
    Opened by PID 918
```

If the connection fails, check the configuration of the router, which is usually accessible using a web browser. Also, examine the output of pptp and the contents of the log file, /var/log/ppp.log for clues.

Chapter 31. Electronic Mail

31.1. Synopsis

"Electronic Mail", better known as email, is one of the most widely used forms of communication today. This chapter provides a basic introduction to running a mail server on FreeBSD, as well as an introduction to sending and receiving email using FreeBSD. For a complete coverage of this subject, refer to the books listed in Bibliography.

This chapter covers:

- Which software components are involved in sending and receiving electronic mail.
- How to configure DragonFly Mail Agent.
- Where basic Sendmail configuration files are located in FreeBSD.
- The difference between remote and local mailboxes.
- How to install and configure an alternate Mail Transfer Agent, replacing DragonFly Mail Agent or Sendmail.
- How to troubleshoot common mail server problems.
- How to configure Sendmail to only send mail.
- How to configure SMTP authentication for added security in Sendmail.
- How to install and use a Mail User Agent, such as mutt, to send and receive email.
- How to download mail from a remote POP or IMAP server.
- How to automatically apply filters and rules to incoming email.

31.2. Mail Components

There are five major parts involved in an email exchange: the Mail User Agent (MUA), the Mail Transfer Agent (MTA), a mail host, a remote or local mailbox, and DNS. This section provides an overview of these components.

Mail User Agent (MUA)

The Mail User Agent (MUA) is an application which is used to compose, send, and receive emails. This application can be a command line program, such as the built-in `mail` utility or a third-party application from the Ports Collection, such as alpine, elm, or mutt. Dozens of graphical programs are also available in the Ports Collection, including Claws Mail, Evolution, and Thunderbird. Some organizations provide a web mail program which can be accessed through a web browser. More information about installing and using a MUA on FreeBSD can be found in Mail User Agents.

Mail Transfer Agent (MTA)

The Mail Transfer Agent (MTA) is responsible for receiving incoming mail and delivering outgoing mail. Starting with FreeBSD version 14.0, the default MTA is DragonFly Mail Agent (dma(8)); in earlier versions, it is sendmail(8). Other MTAs, including Exim, Postfix, and qmail,

may be installed to replace the default MTA.

Mail Host and Mailboxes

The mail host is a server that is responsible for delivering and receiving mail for a host or a network. The mail host collects all mail sent to the domain and stores it either in the default `mbox` or the alternative Maildir format, depending on the configuration. Once mail has been stored, it may either be read locally using a MUA or remotely accessed and collected using protocols such as POP or IMAP. If mail is read locally, a POP or IMAP server does not need to be installed.

Domain Name System (DNS)

The Domain Name System (DNS) and its daemon named(8) play a large role in the delivery of mail. In order to deliver mail from one site to another, the MTA will look up the remote site in DNS to determine which host will receive mail for the destination. This process also occurs when mail is sent from a remote host to the MTA.

31.3. DragonFly Mail Agent (DMA)

DragonFly Mail Agent (DMA) is the default MTA in FreeBSD starting with version 14.0. dma(8) is a small Mail Transport Agent (MTA), designed for home and office use. It accepts mails from locally installed Mail User Agents (MUA) and delivers the mails either locally or to a remote destination. Remote delivery includes several features like TLS/SSL support and SMTP authentication.

dma(8) is not intended as a replacement for real, big MTAs like sendmail(8) or postfix(1). Consequently, dma(8) does not listen on port 25 for incoming connections.

31.3.1. Configuring DragonFly Mail Agent (DMA)

DMA comes with a default configuration that will be suitable for many deployments. Custom settings are defined in /etc/dma/dma.conf, and SMTP authentication is configured in /etc/dma/auth.conf.

31.3.1.1. Using DMA to Route Outgoing Mail through Gmail (STARTTLS:SMTP example)

This example /etc/dma/dma.conf can be used to send mail using Google's SMTP servers.

```
SMARTHOST smtp.gmail.com
PORT 587
AUTHPATH /etc/dma/auth.conf
SECURETRANSFER
STARTTLS
MASQUERADE username@gmail.com
```

Authentication can be set with one line in /etc/dma/auth.conf:

```
username@gmail.com|smtp.gmail.com:password
```



With enabled 2-factor authentication a generated application-specific password as the normal login password will be rejected. See Google documentation for more information on app-specific passwords.

Execute the following command to test the configuration:

```
% echo this is a test | mail -v -s testing-email username@gmail.com
```

31.3.1.2. Using DMA to Route Outgoing Mail through Fastmail (SSL/TLS example)

This example /etc/dma/dma.conf can be used to send mail using Fastmail's SMTP servers.

```
SMARTHOST smtp.fastmail.com
PORT 465
AUTHPATH /etc/dma/auth.conf
SECURETRANSFER
MAILNAME example.server.com
```

Authentication can be set with one line in /etc/dma/auth.conf:

```
username@fastmail.com|smtp.fastmail.com:password
```

Execute the following command to test the configuration:

```
% echo this is a test | mail -v -s testing-email username@fastmail.com
```

31.3.1.3. Using DMA to Route Outgoing Mail through a Custom Mail Host

This example /etc/dma/dma.conf can be used to send mail using a custom mail host.

```
SMARTHOST mail.example.org
PORT 587
AUTHPATH /etc/dma/auth.conf
SECURETRANSFER
STARTTLS
```

Authentication can be set with one line in /etc/dma/auth.conf:

```
username@example.org|mail.example.org:password
```

Execute the following command to test the configuration:

```
% echo this is a test | mail -v -s testing-email username@example.org
```

31.4. Sendmail

Sendmail is a venerable and versatile Mail Transfer Agent (MTA) with a long history in UNIX® and UNIX-like systems. It was a part of the FreeBSD base system until FreeBSD 13, offering robust email transport capabilities, extensive customization options, and support for complex routing and filtering.

31.4.1. Configuration Files

The configuration files for Sendmail are located in /etc/mail/.

/etc/mail/access

This access database file defines which hosts or IP addresses have access to the local mail server and what kind of access they have. Hosts listed as `OK`, which is the default option, are allowed to send mail to this host as long as the mail's final destination is the local machine. Hosts listed as `REJECT` are rejected for all mail connections. Hosts listed as `RELAY` are allowed to send mail for any destination using this mail server. Hosts listed as `ERROR` will have their mail returned with the specified mail error. If a host is listed as `SKIP`, Sendmail will abort the current search for this entry without accepting or rejecting the mail. Hosts listed as `QUARANTINE` will have their messages held and will receive the specified text as the reason for the hold.

Examples of using these options for both IPv4 and IPv6 addresses can be found in the FreeBSD sample configuration, /etc/mail/access.sample:

To configure the access database, use the format shown in the sample to make entries in /etc/mail/access, but do not put a comment symbol (`#`) in front of the entries. Create an entry for each host or network whose access should be configured. Mail senders that match the left side of the table are affected by the action on the right side of the table.

Whenever this file is updated, update its database and restart Sendmail:

```
# makemap hash /etc/mail/access < /etc/mail/access
# service sendmail restart
```

/etc/mail/aliases

This database file contains a list of virtual mailboxes that are expanded to users, files, programs, or other aliases. Here are a few entries to illustrate the file format:

```
root: localuser
ftp-bugs: joe,eric,paul
bit.bucket:  /dev/null
procmail: "|/usr/local/bin/procmail"
```

The mailbox name on the left side of the colon is expanded to the target(s) on the right. The first entry expands the `root` mailbox to the `localuser` mailbox, which is then looked up in the /etc/mail/aliases database. If no match is found, the message is delivered to `localuser`. The second entry shows a mail list. Mail to `ftp-bugs` is expanded to the three local mailboxes `joe`, `eric`, and `paul`. A remote mailbox could be specified as *user@example.com*. The third entry shows how to write mail to a file, in this case /dev/null. The last entry demonstrates how to send mail to a program, /usr/local/bin/procmail, through a UNIX® pipe. Refer to aliases(5) for more information about the format of this file.

Whenever this file is updated, run `newaliases` to update and initialize the aliases database.

/etc/mail/sendmail.cf

This is the master configuration file for Sendmail. It controls the overall behavior of Sendmail, including everything from rewriting email addresses to printing rejection messages to remote mail servers. Accordingly, this configuration file is quite complex. Fortunately, this file rarely needs to be changed for standard mail servers.

The master Sendmail configuration file can be built from m4(1) macros that define the features and behavior of Sendmail. Refer to /usr/src/contrib/sendmail/cf/README for some of the details.

Whenever changes to this file are made, Sendmail needs to be restarted for the changes to take effect.

/etc/mail/virtusertable

This database file maps mail addresses for virtual domains and users to real mailboxes. These mailboxes can be local, remote, aliases defined in /etc/mail/aliases, or files. This allows multiple virtual domains to be hosted on one machine.

FreeBSD provides a sample configuration file in /etc/mail/virtusertable.sample to further demonstrate its format. The following example demonstrates how to create custom entries using that format:

```
root@example.com                root
postmaster@example.com          postmaster@noc.example.net
@example.com                    joe
```

This file is processed in a first match order. When an email address matches the address on the left, it is mapped to the local mailbox listed on the right. The format of the first entry in this example maps a specific email address to a local mailbox, whereas the format of the second entry maps a specific email address to a remote mailbox. Finally, any email address from `example.com` which has not matched any of the previous entries will match the last mapping and be sent to the local mailbox `joe`. When creating custom entries, use this format and add them to /etc/mail/virtusertable. Whenever this file is edited, update its database and restart Sendmail:

```
# makemap hash /etc/mail/virtusertable < /etc/mail/virtusertable
# service sendmail restart
```

/etc/mail/relay-domains

In a default FreeBSD installation, Sendmail is configured to only send mail from the host it is running on. For example, if a POP server is available, users will be able to check mail from remote locations but they will not be able to send outgoing emails from outside locations. Typically, a few moments after the attempt, an email will be sent from `MAILER-DAEMON` with a `5.7 Relaying Denied` message.

The most straightforward solution is to add the ISP's FQDN to /etc/mail/relay-domains. If multiple addresses are needed, add them one per line:

```
your.isp.example.com
other.isp.example.net
users-isp.example.org
www.example.org
```

After creating or editing this file, restart Sendmail with `service sendmail restart`.

Now any mail sent through the system by any host in this list, provided the user has an account on the system, will succeed. This allows users to send mail from the system remotely without opening the system up to relaying SPAM from the Internet.

31.5. Changing the Mail Transfer Agent

Starting with FreeBSD version 14.0, dma(8) is the default MTA, and before 14.0, the default MTA is sendmail(8). However, the system administrator can change the system's MTA. A wide choice of alternative MTAs is available from the `mail` category of the FreeBSD Ports Collection.



If the default's outgoing mail service is disabled, it is important that it is replaced with an alternative mail delivery system. Otherwise, system functions such as periodic(8) will be unable to deliver their results by email. Many parts of the system expect a functional MTA. If applications continue to use the default binaries to try to send email after they are disabled, mail could go into an inactive queue and never be delivered.

31.5.1. Replacing Sendmail with Other MTA

In order to completely disable sendmail(8) execute the following commands:

```
# sysrc sendmail_enable="NO"
# sysrc sendmail_submit_enable="NO"
# sysrc sendmail_outbound_enable="NO"
# sysrc sendmail_msp_queue_enable="NO"
```

To only disable sendmail(8)'s incoming mail service execute the following command:

```
# sysrc sendmail_enable="NO"
```

Then stop the sendmail(8) service:

```
# service sendmail onestop
```

Some extra configuration is needed as sendmail(8) is so ubiquitous that some software assumes it is already installed and configured. Check /etc/periodic.conf and make sure that these values are set to NO. If this file does not exist, create it with these entries:

```
daily_clean_hoststat_enable="NO"
daily_status_mail_rejects_enable="NO"
daily_status_include_submit_mailq="NO"
daily_submit_queuerun="NO"
```

The next step is to install another MTA, dma(8) will be used in this example. As pointed above, dma(8) is the default MTA in FreeBSD starting with version 14.0. Therefore, it is only necessary to install it from the ports when using a previous version.

To install it execute the following command:

```
# pkg install dma
```

Perform the configuration as indicated in Configuring DragonFly Mail Agent (DMA).

Then change all the entries in the file /etc/mail/mailer.conf to dma(8):

```
# Execute the "real" sendmail program, named /usr/libexec/sendmail/sendmail
#
# If dma(8) is installed, an example mailer.conf that uses dma(8) instead can
# be found in /usr/share/examples/dma
#
sendmail        /usr/local/libexec/dma
mailq           /usr/local/libexec/dma
newaliases      /usr/local/libexec/dma
```



When using the version of dma(8) included in the base system, the paths will change to /usr/libexec/dma.

To ensure that anything in the queue is flushed at boot or before shutdown, execute the following command:

```
# sysrc dma_flushq_enable="YES"
```

Once everything is configured, it is recommended to reboot the system. Rebooting provides the opportunity to ensure that the system is correctly configured to start the new MTA automatically on boot.

31.5.2. Replacing DragonFly Mail Agent (DMA) with Other MTA

As noted above, starting with FreeBSD version 14.0, the default MTA is DMA. In this example, mail/postfix will be used as the alternative MTA.

Before installing mail/postfix some extra configuration is needed. Check /etc/periodic.conf and make sure that these values are set to NO. If this file does not exist, create it with these entries:

```
daily_clean_hoststat_enable="NO"
daily_status_mail_rejects_enable="NO"
daily_status_include_submit_mailq="NO"
daily_submit_queuerun="NO"
```

Then install mail/postfix:

```
# pkg install postfix
```

To start mail/postfix at system boot execute the following command:

```
# sysrc postfix_enable="YES"
```



It is good practice to read the installation message after installing an application. Provides useful information about settings, etc.

If postfix is **not** already activated in /usr/local/etc/mail/mailer.conf execute the following commands:

```
mv /usr/local/etc/mail/mailer.conf /usr/local/etc/mail/mailer.conf.old
install -d /usr/local/etc/mail
install -m 0644 /usr/local/share/postfix/mailer.conf.postfix
/usr/local/etc/mail/mailer.conf
```

When employing SASL, ensure that postfix has access to read the sasldb file. This is accomplished by adding postfix to group mail and making the /usr/local/etc/sasldb* file(s) readable by group mail (this should be the default for new installs).

Once everything is configured, it is recommended to reboot the system. Rebooting provides the opportunity to ensure that the system is correctly configured to start the new MTA automatically on boot.

31.6. Mail User Agents

A MUA is an application that is used to send and receive email. As email "evolves" and becomes more complex, MUAs are becoming increasingly powerful and provide users increased functionality and flexibility. The `mail` category of the FreeBSD Ports Collection contains numerous MUAs. These include graphical email clients such as Evolution or Balsa and console based clients such as mutt or alpine.

31.6.1. mail

mail(1) is the default MUA installed with FreeBSD. It is a console based MUA that offers the basic functionality required to send and receive text-based email. It provides limited attachment support and can only access local mailboxes.

Although mail(1) does not natively support interaction with POP or IMAP servers, these mailboxes may be downloaded to a local `mbox` using an application such as fetchmail or getmail.

In order to send and receive email, run mail(1):

```
% mail
```

The contents of the user's mailbox in /var/mail are automatically read by mail(1). Should the mailbox be empty, the utility exits with a message indicating that no mail could be found. If mail exists, the application interface starts, and a list of messages will be displayed.

Messages are automatically numbered, as can be seen in the following example:

```
Mail version 8.1 6/6/93.  Type ? for help.
"/var/mail/username": 3 messages 3 new
>N  1 root@localhost        Mon Mar  8 14:05  14/510   "test"
 N  2 root@localhost        Mon Mar  8 14:05  14/509   "user account"
 N  3 root@localhost        Mon Mar  8 14:05  14/509   "sample"
```

Messages can now be read by typing `t` followed by the message number.

This example reads the first email:

```
& t 1
Message 1:
From root@localhost  Mon Mar  8 14:05:52 2004
X-Original-To: username@localhost
Delivered-To: username@localhost
To: username@localhost
Subject: test
Date: Mon,  8 Mar 2004 14:05:52 +0200 (SAST)
From: root@localhost (Charlie Root)
```

```
This is a test message, please reply if you receive it.
```

As seen in this example, the message will be displayed with full headers.

To display the list of messages again, press `h`.

If the email requires a reply, press either `R` or `r` mail(1) keys. `R` instructs mail(1) to reply only to the sender of the email, while `r` replies to all other recipients of the message. These commands can be suffixed with the mail number of the message to reply to. After typing the response, the end of the message should be marked by a single `.` on its own line.

An example can be seen below:

```
& R 1
To: root@localhost
Subject: Re: test

Thank you, I did get your email.
.
EOT
```

In order to send a new email, press `m`, followed by the recipient email address. Multiple recipients may be specified by separating each address with the `,` delimiter. The subject of the message may then be entered, followed by the message contents. The end of the message should be specified by putting a single `.` on its own line.

```
& mail root@localhost
Subject: I mastered mail

Now I can send and receive email using mail ... :)
.
EOT
```

While using mail(1), press `?` to display help at any time. Refer to mail(1) for more help on how to use mail(1).



mail(1) was not designed to handle attachments and thus deals with them poorly. Newer MUAs handle attachments in a more intelligent way.

31.6.2. Mutt

Mutt is a powerful MUA, with many features, including:

- The ability to thread messages.
- PGP support for digital signing and encryption of email.
- MIME support.

- Maildir support.
- Highly customizable.

Refer to http://www.mutt.org for more information on Mutt.

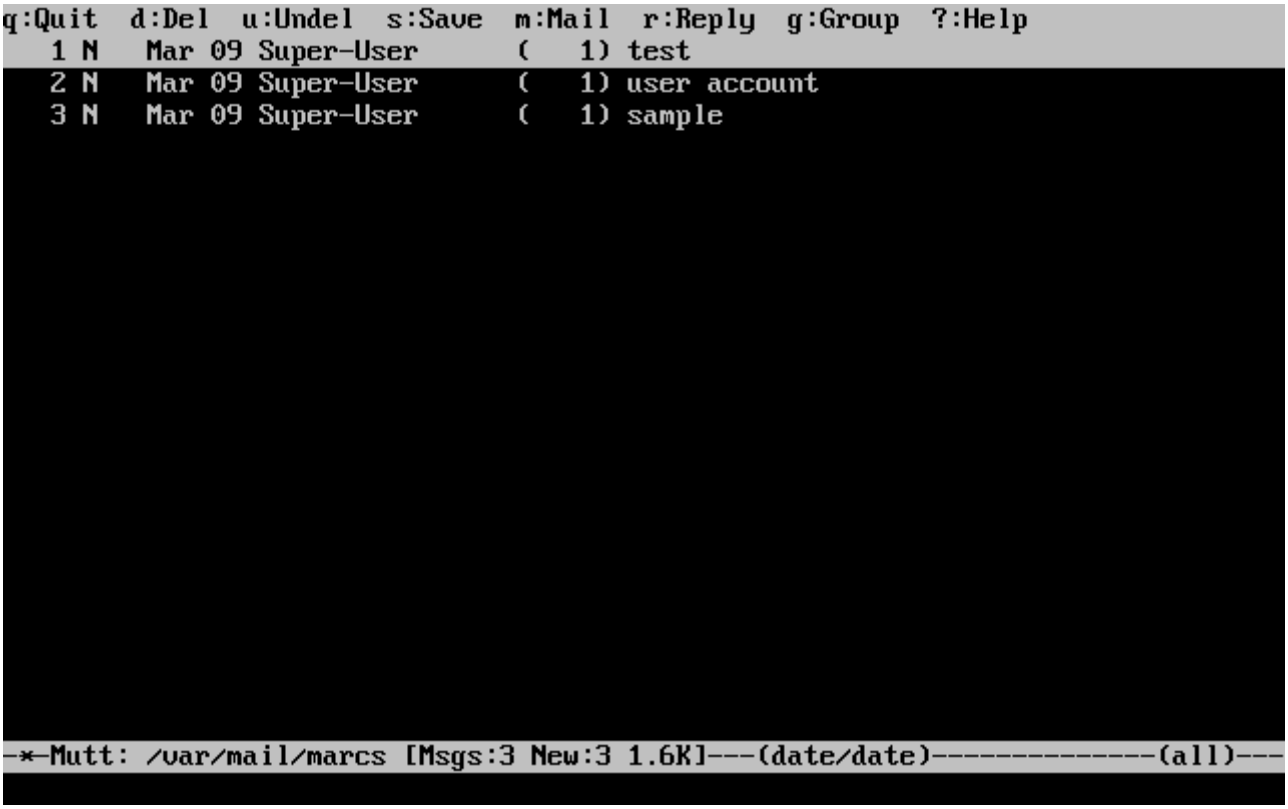


A Mutt fork called NeoMutt is worth mentioning, which brings added features. See more on the NeoMutt website. If NeoMutt was chosen, replace the following command examples from `mutt` to `neomutt`.

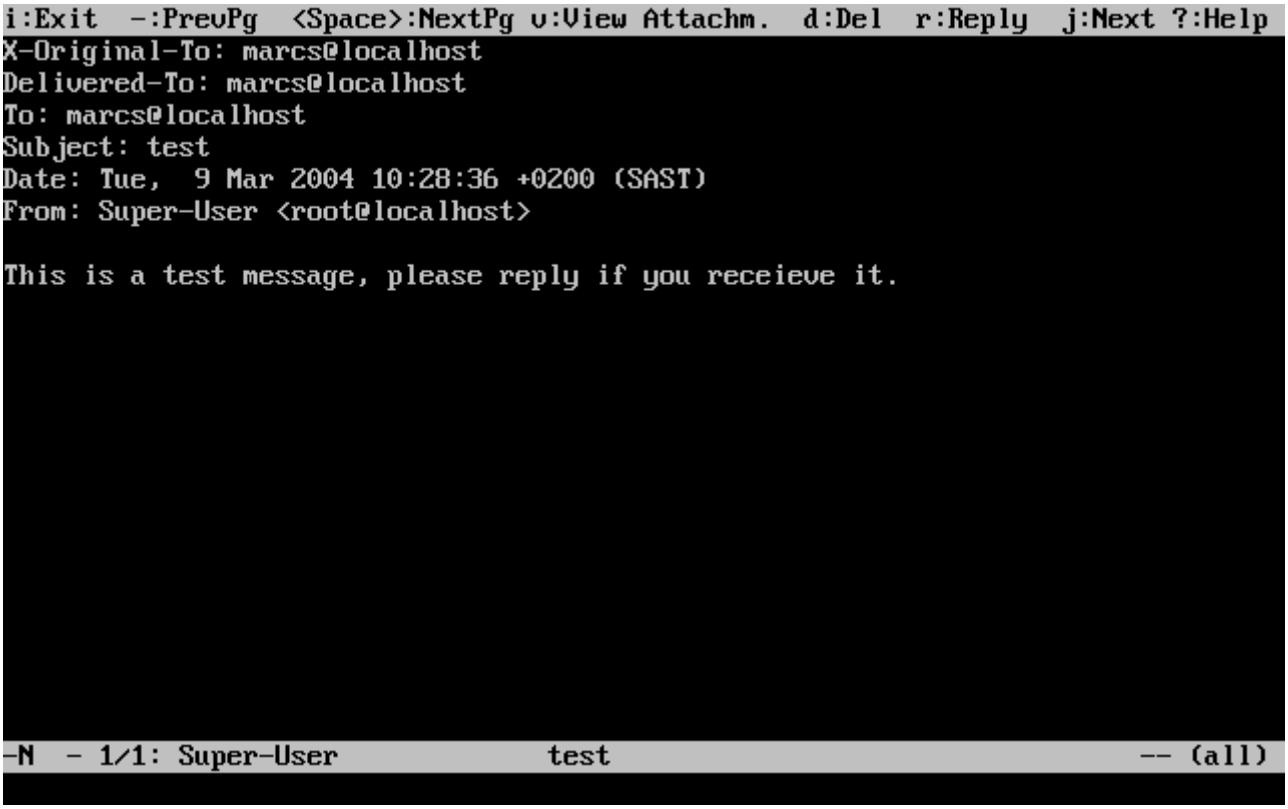
Mutt may be installed using the mail/mutt port. After the port has been installed, Mutt can be started by issuing the following command:

```
% mutt
```

Mutt will automatically read and display the contents of the user mailbox in /var/mail. If no mails are found, Mutt will wait for commands from the user. The example below shows Mutt displaying a list of messages:



To read an email, select it using the cursor keys and press `Enter`. An example of Mutt displaying email can be seen below:

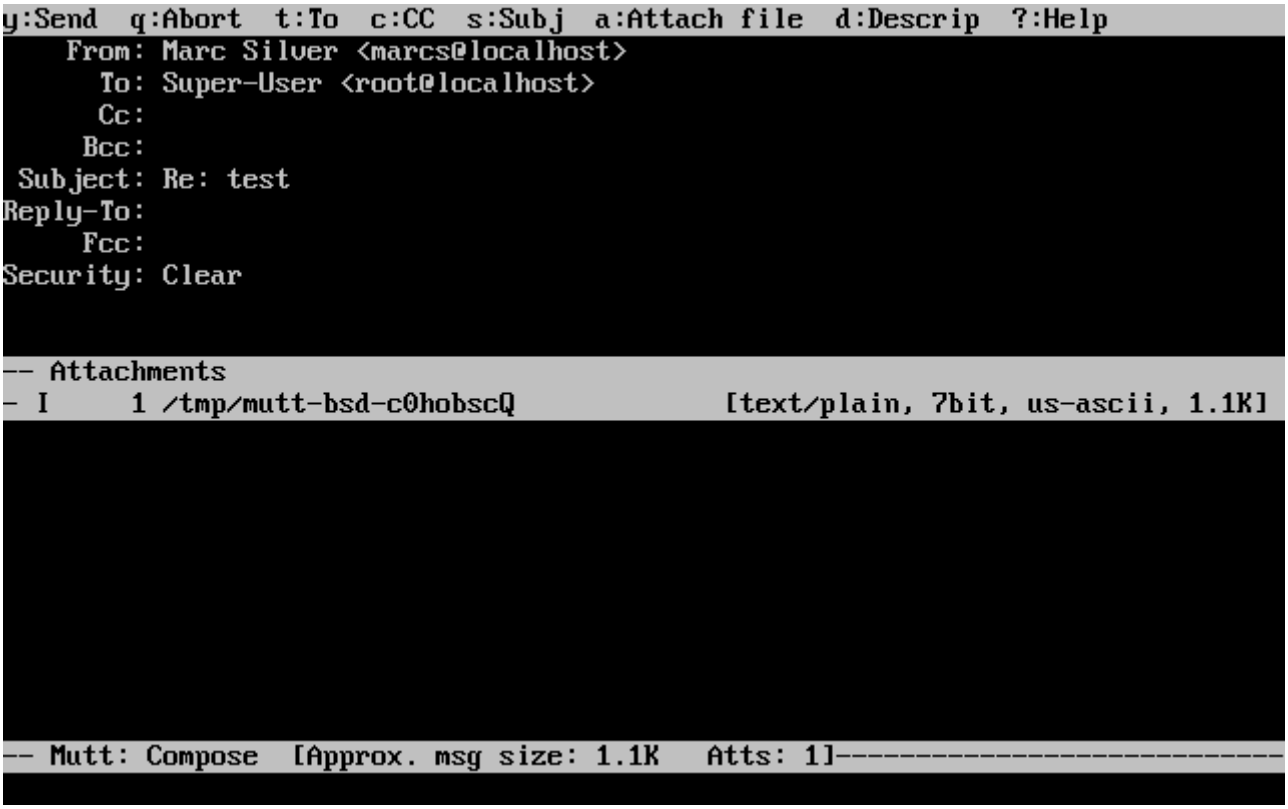


Similar to mail(1), Mutt can be used to reply only to the sender of the message as well as to all recipients. To reply only to the sender of the email, press `r`. To send a group reply to the original sender as well as all the message recipients, press `g`.



By default, Mutt uses the vi(1) editor for creating and replying to emails. Each user can customize this by creating or editing the .muttrc in their home directory and setting the `editor` variable or by setting the `EDITOR` environment variable. Refer to http://www.mutt.org/ for more information about configuring Mutt.

To compose a new mail message, press `m`. After a valid subject has been given, Mutt will start vi(1) so the email can be written. Once the contents of the email are complete, save and quit from `vi`. Mutt will resume, displaying a summary screen of the mail that is to be delivered. In order to send the mail, press `y`. An example of the summary screen can be seen below:



Mutt contains extensive help which can be accessed from most of the menus by pressing `?`. The top line also displays the keyboard shortcuts where appropriate.

31.6.3. alpine

alpine is aimed at a beginner user, but also includes some advanced features.

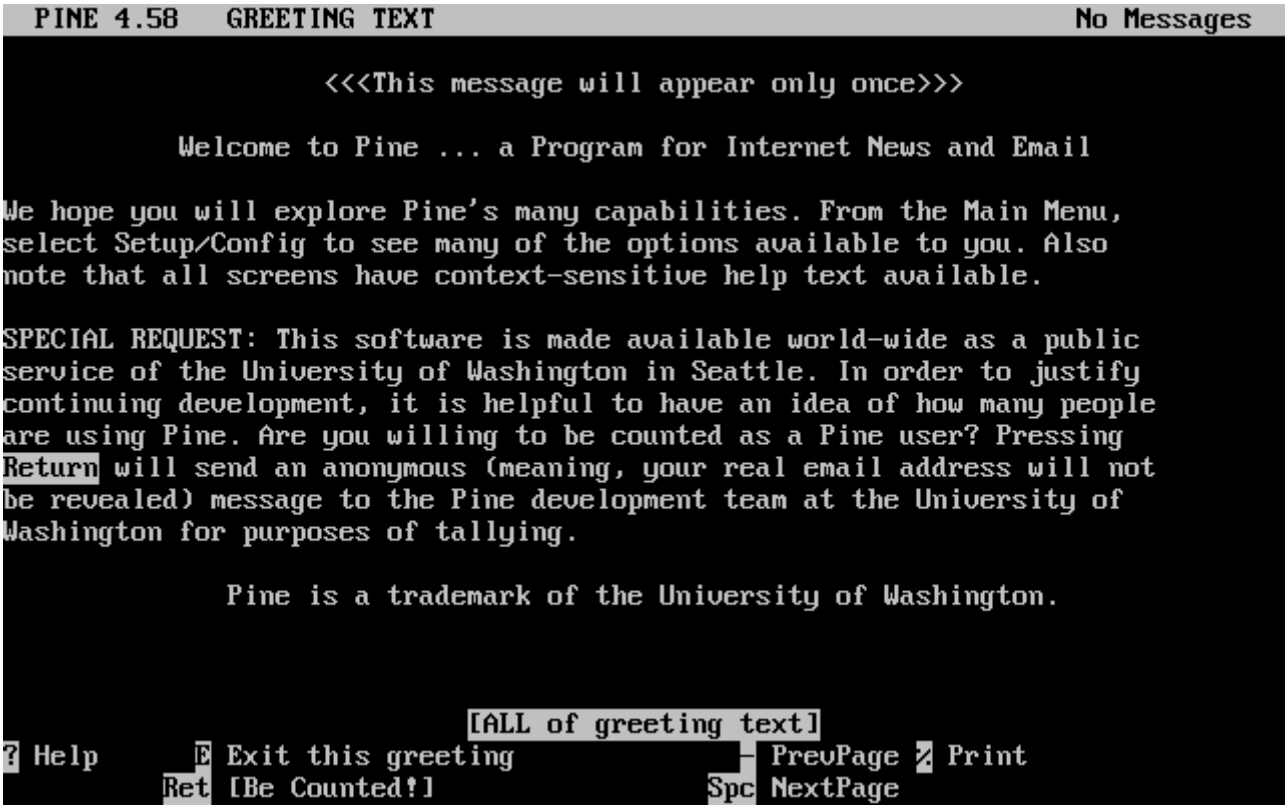


alpine has had several remote vulnerabilities discovered in the past, which allowed remote attackers to execute arbitrary code as users on the local system, by the action of sending a specially-prepared email. While *known* problems have been fixed, alpine code is written in an insecure style and the FreeBSD Security Officer believes there are likely to be other undiscovered vulnerabilities. Users install alpine at their own risk.

The current version of alpine may be installed using the mail/alpine port. Once the port has installed, alpine can be started by issuing the following command:

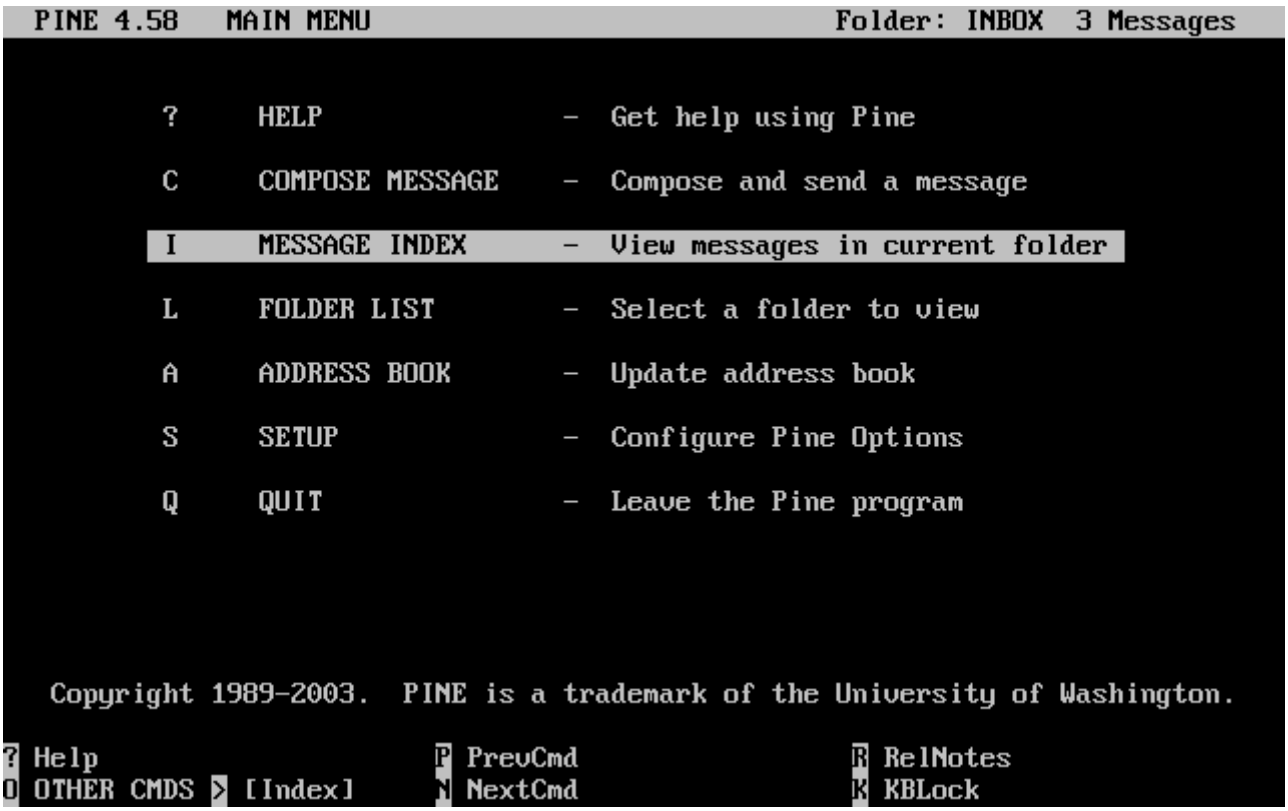
```
% alpine
```

The first time alpine runs, it displays a greeting page with a brief introduction, as well as a request from the alpine development team to send an anonymous email message allowing them to judge how many users are using their client. To send this anonymous message, press `Enter`. Alternatively, press `E` to exit the greeting without sending an anonymous message. An example of the greeting page is shown below:

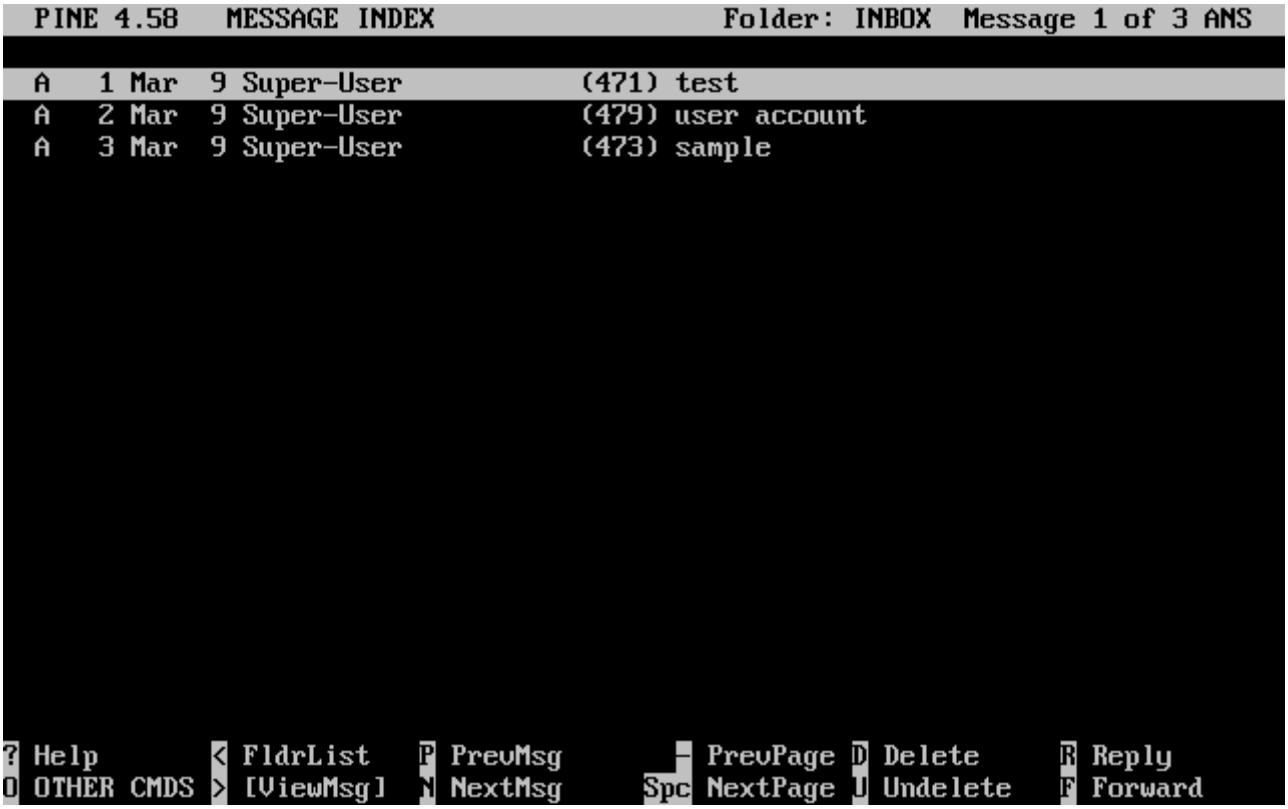


The main menu is then presented, which can be navigated using the cursor keys. This main menu provides shortcuts for the composing new mails, browsing mail directories, and administering address book entries. Below the main menu, relevant keyboard shortcuts to perform functions specific to the task at hand are shown.

The default directory opened by alpine is inbox. To view the message index, press `I`, or select the MESSAGE INDEX option shown below:



The message index shows messages in the current directory and can be navigated by using the cursor keys. Highlighted messages can be read by pressing `Enter`.

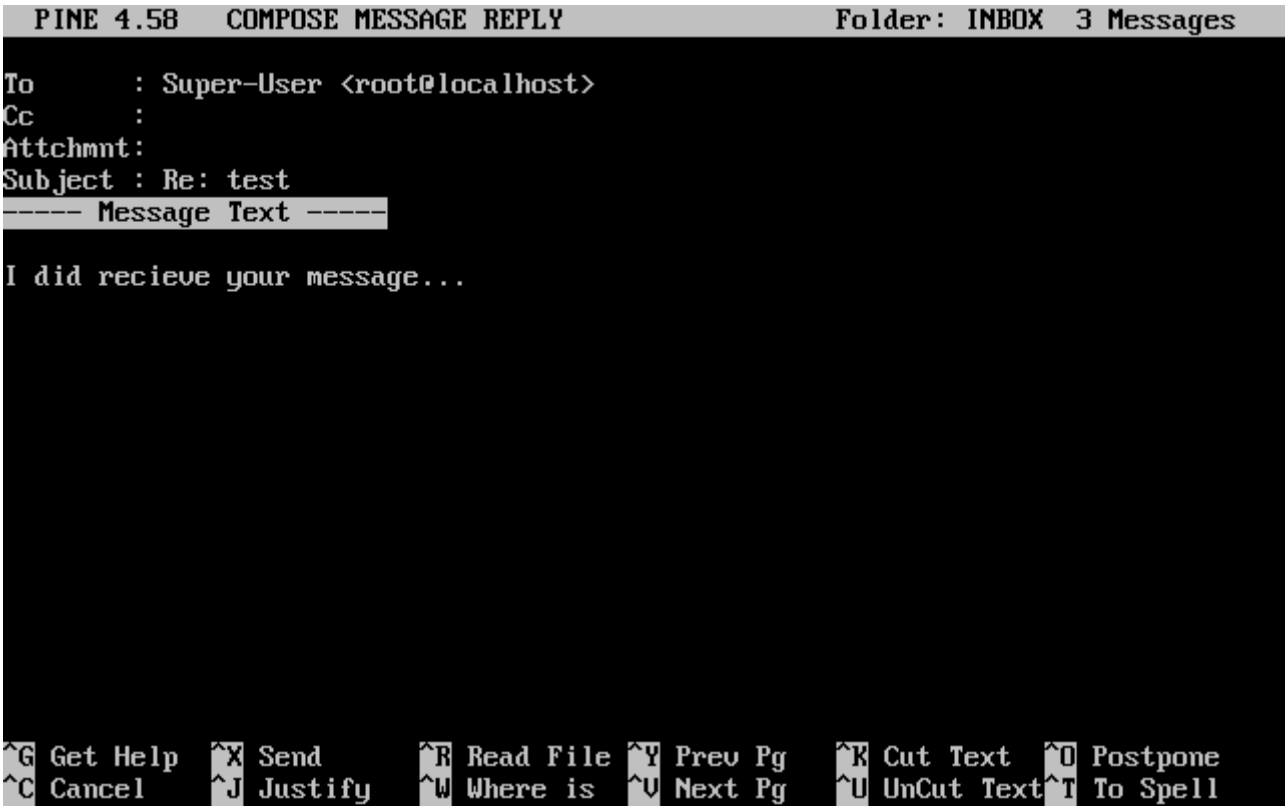


In the screenshot below, a sample message is displayed by alpine. Contextual keyboard shortcuts are displayed at the bottom of the screen. An example of one of a shortcut is `r`, which tells the MUA to reply to the current message being displayed.



Replying to an email in alpine is done using the pico editor, which is installed by default with

alpine. pico makes it easy to navigate the message and is easier for novice users to use than vi(1) or mail(1). Once the reply is complete, the message can be sent by pressing Ctrl + X. alpine will ask for confirmation before sending the message.



alpine can be customized using the SETUP option from the main menu.

31.7. Advanced Topics

This section covers more involved topics such as mail configuration and setting up mail for an entire domain.

31.7.1. Basic Configuration

Out of the box, one can send email to external hosts as long as /etc/resolv.conf is configured or the network has access to a configured DNS server. To have email delivered to the MTA on the FreeBSD host, do one of the following:

- Run a DNS server for the domain.
- Get mail delivered directly to the FQDN for the machine.

In order to have mail delivered directly to a host, it must have a permanent static IP address, not a dynamic IP address. If the system is behind a firewall, it must be configured to allow SMTP traffic. To receive mail directly at a host, one of these two must be configured:

- Make sure that the lowest-numbered MX record in DNS points to the host's static IP address.
- Make sure there is no MX entry in the DNS for the host.

Either of the above will allow mail to be received directly at the host.

Try this:

```
# hostname
```

The output should be similar to the following:

```
example.FreeBSD.org
```

```
# host example.FreeBSD.org
```

The output should be similar to the following:

```
example.FreeBSD.org has address 204.216.27.XX
```

In this example, mail sent directly to yourlogin@example.FreeBSD.org should work without problems, assuming a full-featured MTA is running correctly on `example.FreeBSD.org`. Note that dma(8) does not listen on port 25 for incoming connections and cannot be used in this scenario.

For this example:

```
# host example.FreeBSD.org
```

The output should be similar to the following:

```
example.FreeBSD.org has address 204.216.27.XX
example.FreeBSD.org mail is handled (pri=10) by nevdull.FreeBSD.org
```

All mail sent to `example.FreeBSD.org` will be collected on `nevdull` under the same username instead of being sent directly to the local host.

The above information is handled by the DNS server. The DNS record that carries mail routing information is the mail exchanger record (MX record). If no MX record exists, mail will be delivered directly to the host by way of its IP address.

The MX entry for `freefall.FreeBSD.org` at one time looked like this:

```
freefall        MX  30  mail.crl.net
freefall        MX  40  agora.rdrop.com
freefall        MX  10  freefall.FreeBSD.org
freefall        MX  20  who.cdrom.com
```

`freefall` had many MX entries. The lowest MX number is the host that receives mail directly, if

available. If it is not accessible for some reason, the next lower-numbered host will accept messages temporarily, and pass it along when a lower-numbered host becomes available.

Alternate MX sites should have separate Internet connections in order to be most useful. The ISP can provide this service.

31.7.2. Mail for a Domain

When configuring an MTA for a network, any mail sent to hosts in its domain should be diverted to the MTA so that users can receive their mail on the master mail server.

To make life easiest, a user account with the same *username* should exist on both the MTA and the system with the MUA. Use adduser(8) to create the user accounts.



In addition to adding local users to the host, there are alternative methods known as virtual users. Programs like Cyrus and Dovecot can be integrated into MTAs to handle users, mail storage, and also provide access via POP3 and IMAP.

The MTA must be the designated mail exchanger for each workstation on the network. This is done in the DNS configuration with an MX record:

```
example.FreeBSD.org A   204.216.27.XX       ; Workstation
            MX  10 nevdull.FreeBSD.org  ; Mailhost
```

This will redirect mail for the workstation to the MTA no matter where the A record points. The mail is sent to the MX host.

This must be configured on a DNS server. If the network does not run its own DNS server, talk to the ISP or DNS provider.

The following is an example of virtual email hosting.

Consider a customer with the domain `customer1.org`, where all the mail for `customer1.org` should be sent to `mail.myhost.com`.

The DNS entry should look like this:

```
customer1.org       MX  10  mail.myhost.com
```

An `A` record is *not* needed for `customer1.org` in order to only handle email for that domain. However, running `ping` against `customer1.org` will not work unless an `A` record exists for it.

Tell the MTA which domains and/or hostnames it should accept mail for. Either of the following will work for Sendmail:

- Add the hosts to /etc/mail/local-host-names when using the `FEATURE(use_cw_file)`.
- Add a `Cwyour.host.com` line to /etc/sendmail.cf.

31.7.3. Setting Up to Send Only

There are many instances where one may only want to send mail through a relay. Some examples are:

- The computer is a desktop machine that needs to use programs such as mail(1), using the ISP's mail relay.
- The computer is a server that does not handle mail locally, but needs to pass off all mail to a relay for processing.

While any MTA is capable of filling this particular niche, it can be difficult to properly configure a full-featured MTA just to handle offloading mail. Programs such as Sendmail and Postfix are overkill for this use.

Additionally, a typical Internet access service agreement may forbid one from running a "mail server".

The easiest way to fulfill those needs is to use the dma(8) MTA included in the base system. For systems up to 13.2, need be to installed from ports.

In addition to dma(8), third-party software can be used to achieve the same, like mail/ssmtp.

```
# cd /usr/ports/mail/ssmtp
# make install replace clean
```

Once installed, mail/ssmtp can be configured with /usr/local/etc/ssmtp/ssmtp.conf:

```
root=yourrealemail@example.com
mailhub=mail.example.com
rewriteDomain=example.com
hostname=_HOSTNAME_
```

Use the real email address for `root`. Enter the ISP's outgoing mail relay in place of `mail.example.com`. Some ISPs call this the "outgoing mail server" or "SMTP server".

Make sure to disable Sendmail, including the outgoing mail service.

mail/ssmtp has some other options available. Refer to the examples in /usr/local/etc/ssmtp or the manual page of ssmtp for more information.

Setting up ssmtp in this manner allows any software on the computer that needs to send mail to function properly, while not violating the ISP's usage policy or allowing the computer to be hijacked for spamming.

31.7.4. SMTP Authentication in Sendmail

Configuring SMTP authentication on the MTA provides a number of benefits. SMTP authentication adds a layer of security to Sendmail, and provides mobile users who switch hosts the ability to use

the same MTA without the need to reconfigure their mail client's settings each time.

Install security/cyrus-sasl2 from the Ports Collection. This port supports a number of compile-time options. For the SMTP authentication method demonstrated in this example, make sure that `LOGIN` is not disabled.

After installing security/cyrus-sasl2, edit /usr/local/lib/sasl2/Sendmail.conf, or create it if it does not exist, and add the following line:

```
pwcheck_method: saslauthd
```

Next, install security/cyrus-sasl2-saslauthd and add execute the following command:

```
# sysrc saslauthd_enable="YES"
```

Finally, start the saslauthd daemon:

```
# service saslauthd start
```

This daemon serves as a broker for Sendmail to authenticate against the FreeBSD passwd(5) database. This saves the trouble of creating a new set of usernames and passwords for each user that needs to use SMTP authentication, and keeps the login and mail password the same.

Next, edit /etc/make.conf and add the following lines:

```
SENDMAIL_CFLAGS=-I/usr/local/include/sasl -DSASL
SENDMAIL_LDADD=/usr/local/lib/libsasl2.so
```

These lines provide Sendmail the proper configuration options for linking to cyrus-sasl2 at compile time. Make sure that cyrus-sasl2 has been installed before recompiling Sendmail.

Recompile Sendmail by executing the following commands:

```
# cd /usr/src/lib/libsmutil
# make cleandir && make obj && make
# cd /usr/src/lib/libsm
# make cleandir && make obj && make
# cd /usr/src/usr.sbin/sendmail
# make cleandir && make obj && make && make install
```

This compile should not have any problems if /usr/src has not changed extensively and the shared libraries it needs are available.

After Sendmail has been compiled and reinstalled, edit /etc/mail/freebsd.mc or the local .mc. Many administrators choose to use the output from hostname(1) as the name of .mc for uniqueness.